Oper8

AI-процессы, которые учатся с каждым клиентом

Сергей Липчанский · Асхат Уразбаев

Оглавление

Oper8: строим AI-процессы, которые учатся с каждым клиентом 6
Об авторах . . . 6
Благодарности . . . 8
Дальнейшее чтение . . . 8
Навигатор: у вас проблема — вот глава . . . 8

Часть 1. SELECT 11

Четыре сдвига: что становится возможным 12
Один клиент . . . 12
Каждый контакт учит . . . 13
Продукт-сервис . . . 13
Мгновенность . . . 14
Иерархия: один сдвиг — двигатель, три — результаты . . . 14
Обратная сторона: кого сдвиги уничтожают . . . 14
Пример: ФинСервис . . . 15
Связь с маховиком . . . 16

Ловушка Deploy: почему 74% компаний не видят ROI от AI 17
Парадокс 80/80 . . . 17
Три признака, что вы в ловушке Deploy . . . 18
Что такое Reshape — на пальцах . . . 18
Три теста: Deploy или Reshape? . . . 19
Deploy — правильный ответ, но ловушка всё ещё рядом . . . 20
Эта книга — про Reshape . . . 20

Процесс как продукт: идея, которая запускает трансформацию 21
Два цикла, одна ось . . . 21
Как это выглядит целиком: обработка обращений в «СервисПро» . . . 22
«А что с остальными 80% процессов?» . . . 24
Big Picture: SELECT → FLYWHEEL → SCALE . . . 24

Архитектор и Знание 25
Человек-архитектор . . . 25

Знание как инфраструктура . . . 27
Как два принципа работают вместе . . . 29

SELECT: что менять и насколько глубоко 30
3-Gate Scoring: какой процесс взять первым . . . 30
Deploy / Reshape / Invent: насколько глубоко менять . . . 31
Invent Canvas: если Invent — что именно создавать? . . . 32

Четыре провокации 35
Провокация 1. Frozen Middle — не враг . . . 35
Провокация 2. Метрики опаснее их отсутствия . . . 35
Провокация 3. 30–40% позиций среднего менеджмента исчезнут . . . 36
Провокация 4. Decision Capture окупается даже без AI . . . 36
Пять принципов: итог Части 1 . . . 37
Что дальше . . . 38

Часть 2. FLYWHEEL 40

Как устроен Flywheel 41
Два цикла, одна ось . . . 41
Маппинг: глаголы → шестерёнки → главы . . . 42
Почему каждый оборот раскручивает следующий . . . 42
Что ломается без каждой шестерёнки . . . 43

Knowledge Base: откуда AI знает, как работать 45
Зачем эта шестерёнка . . . 45
Три слоя Knowledge Base . . . 46
Как строить KB: итерация, не водопад . . . 47
Пример: Knowledge Base процесса обработки обращений «СервисПро» . . . 48
Клиентский контур: KB на стыке с клиентом . . . 48
Роль: доменный эксперт . . . 49
Артефакт: Knowledge Base . . . 50
Когда KB не нужна . . . 50
Главная ошибка: «мёртвая» KB . . . 50
Связь со следующей шестерёнкой . . . 51

Autonomy: сколько свободы у AI и где границы 52
Зачем эта шестерёнка . . . 52

Практика 1: Лестница автономии A1–A5 . . . 53
Практика 2: AI Rulebook . . . 55
Практика 3: Dual Run . . . 58
Capability и Permission: важное различие . . . 60
Клиентский контур: автономия по отношению к клиенту . . . 60
Accountability: сквозное ограничение . . . 61
Роль: нет отдельной роли . . . 61
Артефакт: Rulebook . . . 61
Когда Autonomy можно упростить . . . 62
Главная ошибка: размывание границ автономии . . . 62
Связь со следующей шестерёнкой . . . 62

Memory: что решили, почему и оказалось ли верным 64
Зачем эта шестерёнка . . . 64
Практика: Decision Capture . . . 65
Клиентский контур: клиент как главный источник feedback . . . 67
Артефакт: Decision Log . . . 68
Когда Decision Capture не нужен . . . 68
Главный failure mode: петля без обратной связи . . . 69
Связь со следующей шестерёнкой . . . 69

Quality Gate: работает ли? Стало лучше или хуже? 70
Зачем эта шестерёнка . . . 70
Практика: Evals First . . . 71
Клиентский контур: клиентские метрики в eval . . . 73
Роль: Eval Specialist . . . 74
Артефакт: Eval Dataset . . . 74
Когда Quality Gate можно упростить . . . 74
Главный failure mode: «Запустим, потом проверим» . . . 75
Связь со следующей шестерёнкой . . . 75

Metrics: процесс улучшается? Не ломаем ли одно, оптимизируя другое? 76
Зачем эта шестерёнка . . . 76
Практика 1: шесть метрик и антиметрики . . . 77
Практика 2: FinOps — три рычага . . . 80
Клиентский контур: метрики внешнего цикла . . . 81
Роль: AI Process Owner . . . 82

Артефакт: Process Scorecard . . . 82
Когда Metrics можно упростить . . . 82
Главный failure mode: оптимизация без противовеса . . . 83
Замыкание двух циклов . . . 83

Часть 3. SCALE 86

Frozen Middle: шесть рычагов 87
Три механизма сопротивления . . . 87
Шесть рычагов . . . 88
Пример: как «СервисПро» разморозил отдел внедрения . . . 90
Главная ошибка: агрессивный push . . . 91

AI Literacy: четыре уровня 92
Четыре уровня грамотности . . . 92
Кого учить на каком уровне . . . 94
Что не работает . . . 94
Пример: программа «СервисПро» . . . 94
Главная ошибка: обучить и забыть . . . 95

Роли и Role Canvas 96
Три ключевые роли . . . 96
Три дополнительные роли . . . 97
Старые роли не исчезают — мутируют . . . 97
Role Canvas: от «роли изменятся» к конкретному плану . . . 98
Как структурировать команды . . . 99
Пример: оргструктура «СервисПро» через 12 месяцев . . . 99

Часть 4. ВЫХОД 102

Что делать в понедельник 103
Шаг 1. Выберите один процесс . . . 103
Шаг 2. Определите глубину: Deploy, Reshape или Invent . . . 103
Шаг 3. Опишите текущий уровень автономии . . . 104
Шаг 4. Начните Decision Log . . . 104
Шаг 5. Напишите первый eval . . . 104
Шаг 6. Соберите Process Scorecard . . . 104
Что дальше . . . 105

Приложение. КЕЙСЫ 107

Кейс: Логистика последней мили — AI-диспетчер маршрутов 108

1. Контекст 108
2. DRI: Deploy, Reshape или Invent? 108
3. Reshape: что меняется 109
4. Два цикла Flywheel 109
5. Пять шестерёнок 110
6. Экономика 111

Кейс: Рекрутинг — AI-скринер резюме и персональный путь кандидата 112

1. Контекст 112
2. DRI: Deploy, Reshape или Invent? 112
3. Reshape: что меняется 113
4. Два цикла Flywheel 113
5. Пять шестерёнок 114
6. Экономика 115

Кейс: EdTech — адаптивный курс с AI-тьютором 116

1. Контекст 116
2. DRI: Deploy, Reshape или Invent? 116
3. Invent Canvas 116
4. Два цикла Flywheel 118
5. Пять шестерёнок 119
6. Экономика 119

Oper8: строим AI-процессы, которые учатся с каждым клиентом

74% организаций не видят ROI от AI. Проблема не в технологии. Проблема в подходе: компании прикручивают AI к старым процессам вместо того, чтобы перестроить сам процесс. Эта книга — про второй путь.

Восьмёрка в названии — два сцепленных цикла: внешний (клиент учит систему) и внутренний (процесс учится сам). Вместе они образуют маховик — *Data Flywheel*. Возьмите ваш процесс, ваших людей, вашу экспертизу — и перестройте так, чтобы каждый оборот был умнее предыдущего.

Об авторах

У Сергея Липчанского и Асхата Уразбаева есть профессиональная деформация: как только в индустрии что-то устаканивается и все выдыхают — эти двое находят следующую амбразуру и бегут её закрывать.

Десять лет назад они объясняли серьёзным людям в костюмах, что waterfall — это плохая идея, а стикеры на стене — хорошая. Серьёзные люди смотрели с подозрением. Потом те же люди наняли agile-коучей и сделали вид, что всегда так думали.

Теперь история повторяется: только вместо стикеров — агенты, вместо ретроспектив — eval-прогоны, а вместо «давайте попробуем Scrum» — «давайте отдадим этот процесс машине». Реакция серьёзных людей пока примерно та же.

Сергей Липчанский — основатель Кактус.AI, COO ScrumTrek. Двадцать лет ломает компаниям привычный способ работы и удивляется, что его продолжают приглашать.

Асхат Уразбаев — основатель ScrumTrek, притащил Agile в русскоязычный мир. Потом ему стало скучно, и он переключился на AI-трансформации целых операционных моделей. Масштаб амбразуры вырос, привычка бежать на неё — осталась.

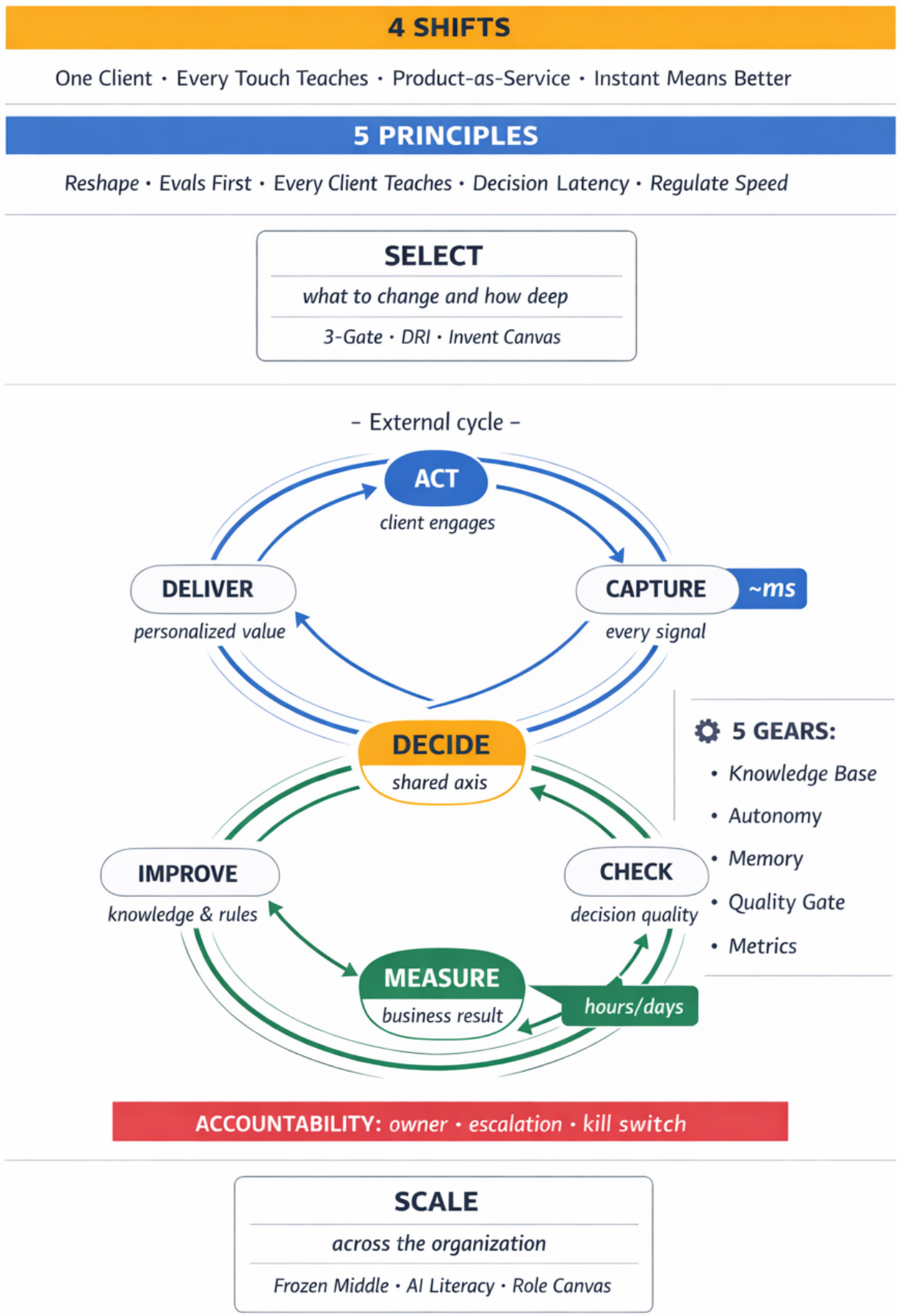

Рис. 1: Big Picture: SELECT → FLYWHEEL → SCALE

Благодарности

Отдельное спасибо людям-экспертам, которые терпеливо вычитывали черновики и не давали авторам улететь в стратосферу: Ольга Качалина, Мария Петрова.

И ИИ-агентам, которые работали над текстом без выходных, перерывов на кофе и претензий по зарплате: Аскар, Кира, Гай, Марат, Игорь.

Дальнейшее чтение

Мы готовим серию практических гайдов — каждый по 10–20 страниц, самостоятельный, с чек-листами и шаблонами:

- *Маршрут внедрения* — 6 фаз от аудита до масштабирования
- *Архитектура* — 8 агентных паттернов, Gate 0, Enable → Embed → Evolve
- *Метрики и FinOps* — три оси ROI, Eval Lifecycle, управление стоимостью
- *Оргструктура* — Team Topologies для AI, все роли, три фазы перестройки
- *Люди и adoption* — Champions Network, продвинутая программа AI Literacy
- *Compliance* — Risk Matrix A1–A5, EU AI Act, дерево решений
- *Discovery* — Process Mining, Task Mining, как найти процессы-кандидаты

Подписывайтесь, чтобы не пропустить:

t.me/aidea4work

Навигатор: у вас проблема — вот глава

Ваша ситуация	Куда читать
«Не понимаем, зачем нам AI — кроме “все внедряют”»	Глава 1.1 — *Четыре сдвига: что становится возможным*
«Запустили 10 пилотов, но ROI нет»	Глава 1.2 — *Ловушка Deploy*
«Не понимаем, какой процесс автоматизировать первым»	Глава 1.5 — *SELECT: что менять и насколько глубоко*
«AI галлюцинирует, не знает наши правила»	Глава 2.1 — *Knowledge Base*

Ваша ситуация	Куда читать
«Боимся дать AI больше свободы»	Глава 2.2 — *Autonomy: лестница A1–A5 и AI Rulebook*
«AI принимает решения, но мы не знаем какие»	Глава 2.3 — *Memory: Decision Capture*
«Качество упало после обновления промпта»	Глава 2.4 — *Quality Gate: Evals First*
«Не можем доказать бизнесу, что AI окупается»	Глава 2.5 — *Metrics: 6 метрик + FinOps*
«Средний менеджмент саботирует внедрение»	Глава 3.1 — *Frozen Middle: 6 рычагов*
«Люди не умеют работать с AI»	Глава 3.2 — *AI Literacy: 4 уровня*
«Не понимаем, кто за что отвечает»	Глава 3.3 — *Роли и Role Canvas*

→ Начнём с главного: что принципиально нового становится возможным. *Глава 1.1 — «Четыре сдвига».*

Часть 1. SELECT

Что становится возможным, как думать и с чего начать

Четыре сдвига: что становится возможным

Прежде чем разбирать *как* — давайте зафиксируем *зачем*.

У AI-трансформации есть неприятное свойство: она легко превращается в бесконечный список «можно было бы...». Чат-бот для HR, copilot для юристов, классификатор тикетов, автоматизация отчётов. Каждый проект звучит полезно. Ни один не меняет бизнес.

Проблема не в проектах. Проблема в том, что нет ответа на вопрос: что принципиально нового становится возможным? Не «быстрее» и не «дешевле» — а что было невозможно вчера и стало возможно сегодня?

Мы выделяем четыре таких сдвига. Каждый из них невозможен без AI — и каждый невозможен без системы, которую мы будем собирать в этой книге.

Один клиент

Единица экономики — конкретный человек.

Банк традиционно считает P&L по продуктовым линейкам: ипотека, автокредиты, карты. Клиент — строчка в CRM, которая попадает в один из 5–7 сегментов. Предложение формируется для сегмента: «мужчины 30–40 лет с доходом выше 150 000 рублей в месяц». Клиент получает то, что получает весь сегмент.

Теперь представьте P&L на уровне одного клиента. Не сегмента — а конкретного Алексея из Казани. Его кредитная история, его паттерн трат, его реакция на прошлые предложения, его предпочтительный канал коммуникации. Уникальная цена, уникальный продукт, уникальный UX.

Почему раньше это было невозможно? Потому что предельные затраты персонального обслуживания были запретительно высокими. Один финансовый советник на 500 клиентов — это «здравствуйте, у нас есть отличное предложение по рефинансированию для вашего сегмента». Один AI-агент на 500 клиентов — это 500 персональных предложений, каждое с учётом контекста.

Предельные затраты персонализации стремятся к нулю. Это меняет всё.

Каждый контакт учит

Клиент находится *внутри* системы обучения, а не снаружи.

В традиционном бизнесе клиент — потребитель. Он получает услугу и уходит. Если компания хочет «учиться на клиентах», она запускает NPS-опрос раз в квартал, нанимает аналитика и через полгода получает отчёт: «клиенты недовольны скоростью обработки заявок». Спасибо, капитан.

В AI-системе каждое действие клиента — автоматический обучающий сигнал. Жалоба улучшает поддержку. Сделка уточняет скоринг. Отказ корректирует рекомендации. Молчание — тоже сигнал. Клиент не «обучает систему» — он *элемент production pipeline*.

Это принципиальная разница. Компания с 10 000 клиентов — это 10 000 параллельных потоков обучения. Каждый день. Автоматически. Без NPS-опроса и без аналитика с отчётом.

Без AI петля обратной связи требовала ручной разметки и анализа. Цикл — месяцы. Теперь *capture* автоматический, а цикл — миллисекунды.

Продукт-сервис

Продукт — динамическое поведение системы.

Что такое банковский продукт сегодня? Приложение с набором экранов. Кнопка «подать заявку», форма, статус, уведомление. Клиент взаимодействует с интерфейсом.

А теперь представьте: банк — не приложение, а финансовый советник 24/7. Он знает ваш контекст (C1), учится на каждом вашем действии (C2) и отвечает мгновенно (C4). Он сам инициирует контакт, когда видит возможность. Не push-уведомление «у нас акция по кредитам», а конкретное предложение: «Алексей, ваш автокредит через 3 месяца закроется. С учётом вашего платёжного поведения мы готовы предложить рефинансирование ипотеки на условиях, которые сэкономят вам 14 000 рублей в месяц. Вот расчёт.»

Продукт ведёт себя так, будто за ним стоит выделенная команда. Интерфейс становится диалогом, а не набором экранов. Граница между «продуктом» и «сервисом» стирается.

Без AI продукт — это вещь, а сервис — это люди. AI стирает эту границу.

Мгновенность

Схлопывание задержки (latency collapse) — когда ответ мгновенный, меняется не UX, а архитектура решений клиента.

Кажется, что мгновенность — это просто «быстрее». Подал заявку — получил ответ за секунду вместо трёх дней. Приятно, но не революционно.

На самом деле мгновенность меняет когнитивное поведение. Instant credit — импульсные покупки, которых раньше не было. Мгновенный ответ на сложный финансовый вопрос — это другая модель exploration: клиент задаёт 15 уточняющих вопросов за 5 минут вместо одного звонка в колл-центр за неделю. Скорость меняет поведение.

Без AI скорость масштабировалась только через найм. 10× больше клиентов — 10× больше операторов. Теперь задержку можно сжать до нуля без потери качества.

Иерархия: один сдвиг — двигатель, три — результаты

У четырёх сдвигов есть иерархия.

C2 (каждый контакт учит) — это двигатель. Без петли обучения C1 невозможен: нечем персонализировать. C3 остаётся маркетинговым лозунгом: нет контекста для консьержа. C4 превращается в быстрые, но одинаково плохие ответы.

C2 — двигатель. C1, C3, C4 — результаты, которые двигатель обеспечивает.

Это важно для практики: если ваша AI-система не замыкает петлю обучения на каждом клиентском контакте — остальные три сдвига останутся красивыми слайдами.

Обратная сторона: кого сдвиги уничтожают

Четыре сдвига — возможность и угроза одновременно для любого бизнеса, который не успел перестроиться. Угроза бьёт не по отраслям, а по бизнес-моделям.

Модель 1: продажа времени. Юрист готовит due diligence за 200 часов. AI-система делает то же за 4. Клиент не готов платить в 50 раз дороже за час только потому, что результат «такой же качественный». Консалтинг, аудит, аутсорс разработки, дизайн-агентства — любой бизнес, где цена привязана к часам, оказывается под давлением.

Когда коллега говорит «я потратил 15 минут на то, на что вы потратили два дня» — это сигнал, что модель ценообразования сломалась.

Модель 2: информационная асимметрия. Финансовый советник, риелтор, рекрутер зарабатывают на том, что «мы знаем, а вы нет». Когда клиент может получить персонализированный анализ сам (C1 + C4) — посредник теряет ценность. Выживают те, кто перестаёт продавать информацию и начинает продавать суждение.

Модель 3: ручная маршрутизация потока. Колл-центры, модерация контента, первичный скрининг резюме, бухгалтерский ввод — любой процесс, где человек стоит между входом и выходом и «маршрутизирует» поток. AI не ускоряет маршрутизатора — он его заменяет.

Вопрос для вашей организации не «внедрять ли AI». Вопрос — сколько времени до того, как конкурент или новый игрок применит эти сдвиги *против вас*. Четыре сдвига произойдут в вашей отрасли. Единственный выбор — быть тем, кто их запускает, или тем, кто их переживает.

Пример: ФинСервис

ФинСервис — средний банк, 30 000 кредитных заявок в месяц. Посмотрим, как четыре сдвига меняют их кредитный процесс.

Было (до сдвигов). Заявка попадает в очередь. Скоринговая модель даёт балл. Андеррайтер проверяет документы, звонит работодателю, принимает решение. Клиент ждёт 3–5 рабочих дней. Все клиенты проходят один и тот же конвейер. Одобрили — получи стандартные условия для своей категории.

Стало (после сдвигов).

Один клиент. Заёмщик Алексей получает не «условия для сегмента», а персональное предложение: ставка учитывает его кредитную историю, паттерн трат по карте ФинСервиса и даже то, что он стабильно платит ЖКХ через приложение банка. P&L считается не по «автокредитам в целом», а по Алексею конкретно.

Каждый контакт учит. Алексей отклонил первое предложение — это сигнал. Система запомнила: ставка была слишком высокой для его ожиданий. Через неделю Алексей снова открыл калькулятор — ещё сигнал: интерес сохраняется. Каждое действие автоматически кормит модель, которая обслуживает следующего заёмщика с похожим профилем.

Продукт-сервис. ФинСервис — не приложение с кнопкой «подать заявку». Это

финансовый агент, который сам говорит Алексею: «Через 3 месяца у вас закрывается автокредит. Я посчитал: если направить эти платежи на досрочное погашение ипотеки, вы сэкономите 840 000 рублей за 5 лет. Показать расчёт?»

Мгновенность. Алексей задаёт вопрос: «А если я возьму рефинансирование и при этом увеличу ежемесячный платёж на 5000?» Ответ — через 2 секунды. Не «мы рассмотрим ваш запрос в течение 3 рабочих дней», а здесь и сейчас, с точными цифрами. Алексей задаёт ещё 8 вопросов за 10 минут. Раньше каждый из этих вопросов требовал звонка.

Связь с маховиком

Четыре сдвига — это не абстрактная цель. Это внешний эффект Data Flywheel — маховика, который мы будем собирать на протяжении всей книги.

Внутри маховик крутит пять шестерёнок: базу знаний, автономию, память, контроль качества, метрики. Снаружи — каждый клиентский контакт кормит память и обогащает базу знаний. Чем больше клиентов — тем быстрее маховик.

Это эффект сложного процента на данных. Конкурент может скопировать вашу архитектуру, но не 50 000 размеченных прогонов с обратной связью. Четыре сдвига *происходят* сами, когда маховик набирает обороты.

Но прежде чем собирать маховик — нужно разобраться с главной ловушкой, в которую попадают 74% компаний.

→ Следующая глава: «*Ловушка Deploy: почему 74% компаний не видят ROI от AI*»

Ловушка Deploy: почему 74% компаний не видят ROI от AI

Представьте: вы — VP по операциям в крупной B2B IT-компании. Год назад совет директоров утвердил бюджет на «AI-трансформацию». Вы купили лицензии, наняли интеграторов, запустили 12 пилотов. Сейчас — промежуточный отчёт. На слайдах всё красиво: чат-бот отвечает на 40% вопросов клиентов, copilot помогает юристам писать договоры на 30% быстрее, AI-агент обрабатывает типовые обращения поддержки.

А потом финансовый директор задаёт один вопрос: «Где это в P&L?»

И вы понимаете, что ответа нет. Расходы выросли — лицензии, API-вызовы, интеграция. А выручка — нет. Людей не стало меньше. Процессы те же, просто с AI-обвеской. Чат-бот отвечает быстро, но на те же вопросы, что и раньше. Юристы пишут быстрее — но договоров больше не стало. AI-агент обрабатывает типовые тикеты автоматически — но это были самые простые обращения, которые операторы и так рассеивали за 20 минут.

Звучит знакомо? Вы в хорошей компании: по данным McKinsey (2024), 74% организаций не видят значимого ROI от AI-инициатив.

Парадокс 80/80

Проблема не в AI. Проблема в подходе.

Большинство компаний «внедряют AI» так: берут существующий процесс — как он есть, со всеми костылями, ручными передачами и Excel-файлами — и прикручивают к нему чат-бот, copilot или автоматизацию. Это подход *Deploy*: добавить AI к тому, что уже работает.

В среде системных архитекторов это называют *paving the cowpath* — асфальтирование коровьей тропы. Тропа стала ровнее, но маршрут по-прежнему идёт через болото. Историческая параллель: при электрификации фабрики просто заменяли паровой двигатель на электромотор, сохраняя запутанную систему ременных передач. Скачок производительности случился десятилетия спустя — когда перестроили саму фабрику.

Deploy даёт +10–20% эффективности. Неплохо. Это тюнинг, а не трансформация.

Настоящий ROI лежит в другом подходе — *Reshape*. Это когда вы не прикручиваете AI к старому процессу, а перестраиваете процесс с нуля, исходя из того, что AI умеет.

Другие роли, другие данные, другая логика принятия решений, другой клиентский опыт. Reshape даёт +30–50% и выше — потому что меняет саму структуру создания ценности.

> Принцип: Перестрой, не автоматизируй. Не ускоряй старый процесс — спроектируй новый. Правильный вопрос: *что становится возможным впервые?*

И вот парадокс: по данным Boston Consulting Group, 80% реального ROI от AI лежит в Reshape и *Invent* (создание новых бизнес-моделей). Но 80% корпоративных бюджетов уходит на Deploy-инфраструктуру: лицензии, интеграции, пилоты-витрины.

Почему? Потому что Deploy — понятно, безопасно и можно показать на квартальном отчёте. А Reshape — страшно, непонятно и никто не объяснил, как это делать пошагово.

Три признака, что вы в ловушке Deploy

Проверьте себя:

Признак 1: AI ускоряет, но не меняет. Тот же процесс, те же люди, те же решения — просто быстрее. Юрист по-прежнему проверяет каждый договор. Оператор по-прежнему отвечает на каждый сложный вопрос. AI — дорогой помощник, а не новая операционная модель.

Признак 2: «Сэкономленное время» — фантом. Отчёт говорит: «Сэкономили 2 000 часов в месяц». Но куда они делись? Людей столько же, зарплаты те же, выручка та же. В P&L этих часов нет — потому что нет механизма, который превращает «время» в деньги или новые возможности.

Признак 3: пилоты не переходят в прод. У вас 5–10 пилотов, каждый «успешный», но ни один не масштабировался. Причина: пилот — это wow-demo на 50 примерах. Production — это 50 000 примеров, граничные случаи, интеграция с унаследованными системами и реальные люди, которые не хотят менять привычки.

Что такое Reshape — на пальцах

Разница между Deploy и Reshape проще всего увидеть на примере.

Deploy: компания добавила чат-бот к процессу обработки обращений. Бот читает типовой тикет и говорит оператору: «Это вопрос про расчёты по договору №123. Попробуй решение из базы знаний — абзац 5». Оператор смотрит, согласен (или пишет что-то своё), отправляет ответ. Процесс тот же. Оператор делает то же самое, только с подсказкой.

Reshape: компания перепроектировала весь процесс. AI-агент читает тикет, определяет тип проблемы, ищет решение в базе знаний, учитывает историю этого клиента и SLA договора. На типовые вопросы (60% потока) агент даёт готовый ответ за 4 минуты вместо 2 часов. Каждое решение логируется: входные данные, какие варианты рассматривались, уровень уверенности, результат через неделю (переоткрыл ли клиент тикет). Сложные вопросы (40%) идут к человеку — но не к «оператору», а к *архитектору поддержки*, который проектирует правила эскалации, калибрует агента и отвечает за метрики. Процесс другой. Роли другие. Данные другие.

Через 6 месяцев Reshape-процесс имеет 150 000 решённых тикетов с паттернами решений и обратной связью. Это конкурентное преимущество, которое нельзя купить или скопировать.

Deploy-процесс через 6 месяцев имеет ровно то, что имел в начале: ту же модель, те же правила, тех же людей. Только лицензия дороже.

Три теста: Deploy или Reshape?

Три вопроса помогают определить, нужен ли вашему процессу Reshape. Логика простая: каждый вопрос проверяет одно измерение — роли, данные, маршрут.

Тест 1: роли. Меняется ли функция людей в процессе? Если после внедрения AI оператор делает то же самое, только быстрее — это Deploy. Если оператор перестаёт обрабатывать тикеты и начинает проектировать правила для агента — это Reshape. Простой индикатор: откройте описание должности через полгода после запуска. Если оно не изменилось — вы в Deploy.

Тест 2: данные. Появляются ли данные, которых раньше не было? Deploy использует существующие данные эффективнее. Reshape порождает новые: лог решений с обоснованием, оценки уверенности агента, обратная связь по каждому решению через N дней. Эти данные — топливо для маховика. Если ваш AI-процесс не производит данные, которые улучшают следующий прогон — маховик не крутится.

Тест 3: маршрут. Может ли AI передавать работу дальше без человека-посредника? В Deploy человек остаётся обязательным звеном в каждой передаче: оператор получает подсказку, принимает решение, пересылает коллеге. В Reshape агент маршрутизирует сам: типовой тикет решает, сложный эскалирует напрямую инженеру, VIP-клиента переводит на выделенную линию — без «прочитал, нажал кнопку, переслал».

Два из трёх тестов положительны — перед вами Reshape. Все три плюс новый рынок или продукт — это уже *Invent*. Ноль или один — скорее всего Deploy, и это нормально: не каждый процесс нуждается в перестройке.

Deploy — правильный ответ, но ловушка всё ещё рядом

Deploy — правильный ответ для стандартных процессов, где ни один из трёх тестов не проходит. Если ваш процесс — чистая исполнительская рутина, без вариативности в ролях, без новых данных, с обязательным человеком на каждом этапе — Deploy даст вам вполне достойные +10–20%. Это честный результат. Много компаний будут довольны.

Ловушка — когда Deploy применяют к процессам, где тесты положительны. Менеджмент смотрит на пилот, видит работающий чат-бот или быстрого помощника, думает: «Отлично, масштабируем». Но на самом деле он выбирает лёгкий путь: нужны интеграции, обучение, культурный сдвиг? Сложно. Давайте просто прикрутим AI к тому, что есть, и на этом успокоимся.

Проблема в том, что тесты не обманывают. Если ваш процесс прошёл два из трёх тестов, значит, в нём скрыт потенциал 30–50%+ ROI. Но вы его не достигнете, если выберете Deploy. Вместо этого будете ежегодно отчитываться: «Вложили 5 млн, получили 500 тыс экономии, но переобучение встало». Это не потому, что AI не работает. Это потому, что вы выбрали неправильный путь.

Фокус: перед тем, как запускать пилот, пройдите три теста. Если вы видите шанс на Reshape — берите его. Это сложнее. Но это работает.

Эта книга — про Reshape

Deploy вы можете сделать без книги. Купить лицензии, запустить пилот, показать красивые слайды — для этого достаточно интегратора и бюджета.

Reshape — другая история. Это перестройка процессов, ролей, метрик и культуры. Для этого нужна система: набор принципов, инструментов и практик, которые работают вместе. Мы называем эту систему Data Flywheel — маховик, который делает каждый следующий прогон процесса лучше предыдущего.

Но прежде чем разбирать маховик по шестерёнкам, нужно понять одну ключевую идею, на которой всё стоит.

→ Следующая глава: «*Процесс как продукт: Data Flywheel и два цикла*»

Процесс как продукт: идея, которая запускает трансформацию

Допустим, вы согласны: четыре сдвига — реальная цель, Deploy — ловушка, Reshape — правильный путь. Но «перестроить процесс вокруг AI» звучит красиво на стратсессии и абсолютно бесполезно в понедельник утром. Что конкретно делать?

Ответ умещается в одно предложение:

> Возьмите бизнес-процесс — и превратите его в продукт, который учится с каждым прогоном.

Причём буквально: дайте ему те же атрибуты, которые есть у софтверного продукта — владелец, тесты, лог изменений, метрики и версии. Это конкретная инженерная обвязка.

Два цикла, одна ось

Процесс-как-продукт работает потому, что внутри него крутятся два сцепленных цикла — мы называем их *Data Flywheel*, маховик данных. Если представить это как схему — получается восьмёрка: два кольца, соединённых общей осью.

Внешний цикл — клиент учит систему. Три шага: DELIVER → ACT → CAPTURE.

Система *доставляет* персонализированную ценность клиенту (DELIVER). Клиент *действует* — использует продукт, принимает решение, задаёт вопрос, отказывается от предложения (ACT). Каждое действие — автоматический сигнал, который система *захватывает* (CAPTURE). Этот цикл вращается быстро — миллисекунды на каждый контакт.

Внутренний цикл — процесс учится сам. Четыре шага: CAPTURE → CHECK → MEASURE → IMPROVE.

Захваченные сигналы *проверяются* на качество (CHECK — это evals, тесты, поиск регрессий). Результат *измеряется* в бизнес-метриках (MEASURE — помогает ли это клиентам? Зарабатываем ли мы?). Инсайты *улучшают* базу знаний, правила, модели (IMPROVE). Обновлённое знание возвращается в DELIVER — и внешний цикл крутится точнее. Этот цикл медленнее — часы, дни.

> Принцип: Сначала тесты, потом система. Прежде чем строить AI-агента,

определите, как вы поймёте, что он работает. 20 эталонных кейсов, метрика качества, порог отката.

Принцип: Каждый клиент учит систему. Каждое действие клиента — автоматический обучающий сигнал. Без записи нет памяти, без памяти маховик стоит.

В точке пересечения — DECIDE: AI принимает решения в рамках правил, которые человек задал. Это перекрёстье, где улучшенные знания превращаются в конкретные действия.

Два цикла кормят друг друга через общую ось данных. То, что клиент генерирует во внешнем цикле, становится тем, что процесс оценивает во внутреннем. То, что процесс улучшает во внутреннем, становится тем, что клиент получает во внешнем.

Если вы знакомы с циклами PDCA или HADI — логика та же, но здесь два контура вместо одного: один крутится на каждом клиентском контакте, другой — на каждом цикле оценки. Именно эта связка обеспечивает кумулятивный эффект.

Каждый из семи шагов — DELIVER, ACT, CAPTURE, DECIDE, CHECK, MEASURE, IMPROVE — опирается на одну из пяти «шестерёнок»: Knowledge Base, Autonomy, Memory, Quality Gate, Metrics. Каждую шестерёнку мы подробно разберём в Части 2. Здесь — общая картина на одном примере.

Как это выглядит целиком: обработка обращений в «СервисПро»

В главе 1.1 мы показали четыре сдвига на примере банка «ФинСервис» — кредитные заявки, скоринг, персональные предложения. Теперь возьмём другой тип процесса — клиентскую поддержку. «ФинСервис» показал *что* становится возможным. Теперь посмотрим *как* это работает изнутри: два цикла на одном живом процессе.

Компания «СервисПро» — B2B IT-сервис, 3000 сотрудников, 800+ тикетов в день от корпоративных клиентов. «СервисПро» перестраивает обработку обращений поддержки. Не прикручивает AI к старому процессу (это был бы Deploy), а строит процесс-как-продукт с двумя циклами. Этот кейс пройдёт через всю Часть 2: на нём мы покажем каждую шестерёнку маховика в деле.

Внешний цикл: клиент внутри системы.

DELIVER: AI-агент формирует ответ, учитывая контекст клиента — продукт, SLA, историю обращений, предпочтительный стиль коммуникации. Не шаблонный ответ «для всех», а персонализированный.

ACT: клиент получает ответ и действует. Принял решение? Задал уточняющий

вопрос? Переоткрыл тикет через неделю? Молча закрыл? Каждый исход — сигнал.

CAPTURE: система автоматически записывает всё: текст тикета, какие решения рассматривались, *reasoning* (ход рассуждений агента), *confidence score* (уверенность), а через 14 дней — feedback: клиент вернулся или нет?

Этот цикл крутится 800 раз в день. Каждый тикет учит систему.

Внутренний цикл: процесс учится.

CHECK: 180+ тест-кейсов проверяют качество. Типовые тикеты, нетипичные ситуации, провокационные тесты. При обновлении промпта — прогон всех тестов. Если *eval score* упал — автоматический откат.

MEASURE: владелец процесса Мария смотрит на четыре числа: стоимость решения (*cost-per-resolution*), время до решения (*time-to-resolution*), *eval score* (процент правильных ответов агента на наборе кейсов с известным правильным ответом), *HIR* — доля обращений, требующих вмешательства человека. У каждой метрики — антиметрика: снижаем стоимость — следим за удовлетворённостью клиентов (CSAT), снижаем HIR — следим за процентом переоткрытых тикетов.

IMPROVE: инсайты возвращаются в базу знаний. Новый тип проблемы → новая статья в KB. Частая ошибка агента → обновлённый guardrail. Низкий confidence на определённом типе тикетов → расширенный eval dataset.

DECIDE: на пересечении циклов агент принимает решения в рамках правил. Стартовали на A2 (AI готовит, оператор проверяет). Через 4 месяца данные показали accuracy 96% на типовых тикетах — перевели на A3 (AI решает, человек подтверждает пакетно). Красная линия: проблемы безопасности — всегда на человека.

Результат за 9 месяцев:

Метрика	Было	Стало
Cost-per-resolution	$15	$4
Time-to-resolution	2 часа	4 минуты (типовые)
HIR	100%	12%
Eval score	не было	96%

Но самое важное — не цифры. Самое важное — что через 9 месяцев у Марии 150 000 решённых тикетов с паттернами решений и обратной связью. И во внешнем цикле — 150 000 записей о том, как клиенты реагировали на каждый ответ. Это эффект сложного процента для операционки: конкурент, который стартует сегодня, начнёт с нуля, а СервисПро уже на 150 000-м обороте обоих маховиков.

«А что с остальными 80% процессов?»

Резонный вопрос: не каждый процесс заслуживает PO, 200 evals и структурированного лога. Формализовать всё — это бюрократия 2.0.

Правило порога: если процесс проходит больше 100 прогонов в месяц, влияет на выручку или *compliance* и имеет больше 3 передач между людьми или системами — он кандидат на «продуктовую обвязку». Таких в организации обычно 15–20%.

Остальные 80% не игнорируются — для них работает облегчённый режим: автоматизация на уровне A1–A2, минимальный мониторинг, без выделенного владельца. Это не «второй сорт» — это рациональное распределение инвестиций.

Big Picture: SELECT → FLYWHEEL → SCALE

Вся методика этой книги укладывается в три уровня:

SELECT — стратегический вход. Какой процесс взять? Насколько глубоко менять — Deploy, Reshape или Invent? Если Invent — что именно создавать? Три практики: 3-Gate Scoring, DRI (Deploy / Reshape / Invent), Invent Canvas. Это то, что вы делаете *до* запуска маховика.

FLYWHEEL — операционный уровень. Два сцепленных цикла крутятся на одном процессе. Пять шестерёнок, семь глаголов, три роли, пять артефактов. Практики Control, Learn, Measure живут здесь. Это то, что происходит *каждый день*.

SCALE — организационный выход. Маховик работает на одном процессе — как масштабировать на организацию? Frozen Middle, AI Literacy, Role Canvas. Это то, что вы делаете, когда первый маховик уже крутится.

SELECT → FLYWHEEL → SCALE → *repeat* для следующего процесса.

Часть 1, которую вы сейчас читаете, — про SELECT: зачем, как думать, что менять. Часть 2 разберёт FLYWHEEL: каждую шестерёнку с практиками внутри. Часть 3 — SCALE: масштабирование на всю организацию.

Но прежде чем нырять в шестерёнки — два оставшихся принципа, без которых маховик не заработает.

→ Следующая глава: «*Человек-архитектор и знание как инфраструктура*»

Архитектор и Знание

Два принципа, которые остались. Мы объединяем их в одну главу, потому что они работают в паре: первый отвечает за роль человека, второй — за то, на чём эта роль строится.

Человек-архитектор

> Принцип: Скорость решений, а не скорость работы. Главная единица оптимизации — *decision latency*: как быстро процесс принимает правильное решение.

Три года назад типичный PM тратил 80% времени на логистику — пересылки, согласования, статус-апдейты — и 20% на мышление. Сейчас пропорция переворачивается. AI забирает логистику, а человеку остаётся то, что AI пока делать не умеет: проектировать систему, задавать ей границы и принимать решения в зонах неопределённости.

Это и есть Человек-архитектор. Человек не исполняет задачи — он проектирует систему, которая исполняет задачи, и отвечает за её результат.

На практике это означает три сдвига.

Сдвиг 1: ценность — в решениях, а не в действиях

Раньше специалист поддержки обрабатывал 50 тикетов в день, и его производительность измерялась количеством. В новой модели он принимает 12 решений — какие правила добавить в агента, где поднять порог уверенности, как обработать новый тип обращения — и эти решения снижают стоимость обработки одного обращения на 8%.

Привычка мерить работу в часах уходит — приходит измерение через качество решений. Это болезненный переход: большинство людей в организации определяют себя через действия («я обработал», «я проверил», «я отправил»), а не через проектирование. По нашим наблюдениям, менее 15% ролей в типичной компании на 1000+ человек включают проектирование систем как основную функцию. Остальные — исполнение по заданным процессам. Переход к Human-as-Architect требует не только новых навыков, но и нового самоопределения.

Сдвиг 2: обучение — на суждения, а не на инструменты

Вопрос на собеседовании меняется. Вместо «умеете ли вы работать в Excel» — «как бы вы решили задачу Y при ограничениях Z». Вместо «10 лет опыта в отрасли» — способность калибровать суждения (*judgment*) в новых ситуациях.

Это не значит, что технические навыки больше не нужны. Но навык исполнения становится базовым (его усиливает AI), а навык проектирования и оценки — дифференцирующим.

По данным World Economic Forum (Future of Jobs Report 2025), 44% навыков текущих работников потребуют обновления в ближайшие 5 лет. Это уже происходит.

Сдвиг 3: техническая возможность — это не разрешение

AI может быть технически способен на полную автономию, но бизнес-контекст разрешает ему работать только под присмотром. Архитектор определяет не только что система умеет, но и что ей позволено.

Представьте: AI-агент «СервисПро» технически способен обрабатывать обращения на A5 — полностью автономно. Но политика компании требует, чтобы обращения с критичностью P1 и инциденты безопасности подтверждал человек. Поэтому агент работает на A3 — формирует решение, человек подтверждает. Это ключевое разделение, к которому мы вернёмся в главе про Autonomy.

Слон в комнате

Если человек становится архитектором — что делать с 60–70% сотрудников, чья работа состоит из исполнения? Замолчать этот вопрос — потерять доверие любой C-level аудитории.

Давайте начнём со страшной мысли, которую большинство не говорит вслух.

60–70% ваших сотрудников не станут архитекторами. Это вопрос распределения: не все могут быть архитекторами организации. Но что с ними будет? Ответ может вас испугать.

Они станут частями AI-управляемых процессов. Не человеческий босс, который смотрит в глаза и говорит: «Вот твой проект, покажи результат». Вместо этого — AI-оркестратор, который говорит: «Задача 127, сделай к 15:00, результат проверю автоматически». Задача выдана, результат проверен, обратная связь дана. Человек — исполнитель в системе, которую контролирует и оценивает не директор отдела, а алгоритм.

Уже есть компании, где так работает. Amazon использует AI-систему для управления

работниками на складах — именно так: система выдаёт задачи, автоматически измеряет производительность, генерирует feedback. Toyota экспериментирует с похожим подходом. Это будущее ближе, чем кажется.

Но вот что важно: это не худший исход. Лучше ясные правила, которые установит алгоритм, чем микроменеджер с капризом. Лучше прозрачные метрики производительности, чем политика отдела. И главное — люди, которые работают в таких системах, обычно не против: это даёт чёткость, справедливость и меньше бюрократии.

Страшно не потому, что это плохо. Страшно потому, что это неизбежно. И для большинства вашей организации это будет новая реальность. Они перестанут быть сотрудниками организации в классическом смысле. Они станут агентами в системе — обучаемыми, проверяемыми, заменяемыми.

Честный ответ: Human-as-Architect — это спектр, а не переключатель.

Примерно 60–70% сотрудников переходят на уровень использования — работают с AI-инструментами в рамках своей роли. Юрист не проектирует агента, но проверяет и утверждает его результат. Это доступно большинству и не требует радикальной смены профессии.

15–20% способны дорасти до уровня проектирования — проектировать процессы и evals. Координатор становится process architect, QA-инженер становится eval specialist. Мост между старой и новой ролью критичен: без него человек чувствует, что начинает карьеру с нуля.

Остальные 10–25% — зона честного разговора. Не у всех есть предрасположенность к архитектурному мышлению, и это не приговор, а факт, который лучше признать и помочь людям найти новый путь, чем делать вид, что все справятся.

→ Подробнее о том, как выстроить переход: глава 3.2 «AI Literacy» и глава 3.3 «Роли».

Знание как инфраструктура

> Принцип: Регулируй обороты. Автономия AI растёт через уровни (A1–A5), с границами (*Rulebook*) и безопасным переходом. Каждая метрика имеет противовес.

Вернёмся к «СервисПро» — B2B IT-сервису, на примере которого в предыдущей главе мы разобрали два цикла маховика. Процесс поддержки работает на A3, маховик крутится, метрики растут. А потом увольняется Сергей — старший специалист, единственный человек, который знал все особенности интеграции с API клиента «МегаТех». На следующий день оценка качества (*eval score*) для обращений этого клиента падает с 91% до 64%.

Что пошло не так? Знание Сергея не было формализовано. Оно жило в его голове — и ушло вместе с ним. В терминологии методики: информационная монополия одного эксперта превратилась в единую точку отказа для всего процесса.

Knowledge management пытаются внедрить 20 лет, и 20 лет проигрывают: формализация знаний дорогая, люди документируют из-под палки, а выгода откладывается на неопределённое «потом». AI меняет это уравнение: впервые *capture* становится дешёвым. Автоматическое извлечение решений из переписок, автогенерация summary после созвонов, extraction правил из разрозненных документов — человеку не нужно «записывать», система записывает за него, а человек валидирует.

Это снижает сопротивление формализации на порядок и превращает знание из бюрократической повинности в инфраструктуру, на которой работают агенты.

Три следствия

Документация — это исполняемый код. Промпты, контексты, evals — те же «регламенты», только живые. В AI-native компании документ пишут не для аудита, а потому что он непосредственно определяет поведение агентов. Если промпт плохой — агент выдаёт мусор. Если knowledge base пуст — агент галлюцинирует. Документация перестаёт быть бюрократией и становится частью продукта.

Экспертиза вне системы — это liability. По данным Deloitte (2024), 42% организаций теряли критические знания при уходе ключевых сотрудников. Когда эксперт — единственный носитель знания, это liability. Знание, которое нельзя передать системе, нельзя масштабировать.

Знание устаревает, и это нормально. Автоматически захваченное решение полугодовой давности может быть опаснее отсутствия документации — оно создаёт ложную уверенность. Поэтому знание — инфраструктура, а не архив. Инфраструктуру обслуживают: срок жизни записей (*TTL — Time to Live*) в базе знаний, автоматическая проверка актуальности, ретроспективы по решениям на устаревшем контексте.

Практический тест

Попросите агента: «Подготовь решение для клиента X на базе наших условий.» Если агент не может собрать ответ — проблема не в агенте, а в знании. Либо оно не формализовано, либо недоступно, либо устарело.

Как два принципа работают вместе

Human-as-Architect без знания как инфраструктуры — это архитектор без строительных материалов. Он может проектировать сколько угодно, но агентам не на чем работать.

Знание как инфраструктура без Human-as-Architect — это библиотека без читателей. Данные копятся, но некому решить, что с ними делать, как калибровать, где границы.

В «СервисПро» это выглядит так: Мария (PO, архитектор процесса) проектирует правила и задаёт границы. Сергей (до увольнения) и другие domain experts формализуют знание в KB. Агент работает на пересечении — берёт знание из KB и действует в рамках, которые задал архитектор. Если знание устарело — Мария видит это по метрикам и инициирует обновление. Цикл замкнут.

→ Следующая глава: «SELECT: *что менять и насколько глубоко*»

SELECT: что менять и насколько глубоко

Вы поняли принципы, увидели Flywheel, прониклись идеей «процесс как продукт». Логичный следующий шаг — выбрать первый процесс и начать. И вот тут большинство компаний совершают первую ошибку: берут не тот процесс.

Типичный сценарий: руководитель выбирает процесс, который «очевидно подходит для AI» — потому что о нём все говорят, или потому что CEO увидел demo на конференции. Через 3 месяца пилот «успешен» (на 50 примерах), но масштабировать невозможно — данных нет, люди не готовы, регуляторные ограничения не учли.

Кластер SELECT — это три инструмента, которые решают проблему выбора до того, как вы потратите бюджет.

3-Gate Scoring: какой процесс взять первым

Прежде чем оценивать кандидатов, пропустите каждый через три обязательных gate'а. Если хотя бы один не пройден — процесс не кандидат, какой бы привлекательной ни казалась идея.

Gate 1: связь с измеримым показателем (KPI). Если нельзя сказать «какой показатель изменится и на сколько» — процесс не готов. «Будет лучше» и «команда станет эффективнее» — слишком размыто. «Стоимость обработки одной заявки снизится с $12 до $5» — это показатель.

Gate 2: доступность данных. Готовность данных (*data readiness*) — самый частый тихий убийца AI-инициатив. Данные могут быть в Excel, в головах людей, в переписках. Это не стоп-сигнал, но вам нужно понимать стартовую точку. Если данных вообще нет и взять неоткуда — рано.

Gate 3: результат, на основе которого можно действовать (*actionable output*). Организация должна уметь действовать на основе вывода AI. По данным Gartner (2024), примерно 40% AI-моделей в production не интегрированы в процесс принятия решений — output просто игнорируется. Нет смысла предсказывать отток клиентов, если некому работать с прогнозом.

Три gate'а пройдены? Теперь оцениваем по пяти критериям:

Критерий	Вес	Что оценивается
Бизнес-ценность	25–30%	Экономия, выручка, удовлетворённость клиентов
Техническая реализуемость	20–25%	Доступность данных, сложность интеграции

Критерий	Вес	Что оценивается
Стратегическое соответствие	15–20%	Приоритеты компании, конкурентная дифференциация
Риск и соответствие требованиям	15–20%	Регуляторные требования, этика, репутация
Организационная готовность	10–15%	Наличие людей, готовность к изменениям, спонсор

И три правила, которые превращают число в решение:

Вето-правило: риск и соответствие ниже 2 из 5 при любой бизнес-ценности — автоматический стоп. Организационная готовность ниже 2 (нет спонсора, нет команды) — отложить.

Правило конфликта: высокая ценность (5) + низкая реализуемость (2) — это не Quick Win, а Strategic Bet. Timeline автоматически удваивается.

Порог входа: итоговый балл ниже 3.0 — не кандидат в этом квартале. Пересмотреть через 3 месяца, потому что capabilities AI меняются быстро.

Антипаттерн: «начнём с чего попроще»

Выбирать самый простой процесс для пилота — интуитивно привлекательно и почти всегда неправильно. Простые процессы дают маленький эффект, который никого не убеждает. Лучше выбрать процесс средней сложности с высоким бизнес-impact — достаточно простой для production за 3 месяца и достаточно ценный для заметного результата в P&L.

Deploy / Reshape / Invent: насколько глубоко менять

Вы выбрали процесс. Теперь второй вопрос: что именно с ним делать?

Deploy — добавить AI к существующему процессу. Роли, шаги и логика те же, просто быстрее. Эффект: +10–20%. Уместно, когда процесс уже хорошо работает. Пример: copilot для юристов, который генерирует черновики.

Reshape — перестроить процесс вокруг того, что AI умеет. Другие роли, другие данные, другая логика. Эффект: +30–50% и выше. Уместно, когда текущий процесс — коровья тропа. Пример: СервисПро из предыдущих глав.

Invent — создать то, чего раньше не было. Новый продукт, новый рынок, новая бизнес-модель. Эффект: непредсказуемый, но потенциально кратный. Уместно, когда AI открывает возможности, невозможные без него.

Три RRI-индикатора

RRI (*Reshape Readiness Indicators*) помогают определить глубину:

Индикатор	Вопрос
Автоматизированный handoff	Может ли AI передавать работу дальше без человека-посредника?
Новый тип данных	Появляются ли данные, которых раньше не было?
Изменённая роль	Меняется ли функция человека в процессе?

Ноль или один индикатор — скорее всего Deploy (и это нормально). Два из трёх — Reshape. Все три плюс новый рынок — Invent.

Антипаттерн: Reshape всего подряд

Организация воодушевилась и решила перестроить 15 процессов одновременно. Через 3 месяца — ни один не доведён до production. Правило: один Reshape в production лучше, чем пять на стадии пилота.

Invent Canvas: если Invent — что именно создавать?

DRI-оценка показала потенциал Invent. Все три индикатора положительны, и вы видите новый рынок или продукт. Но «создать новое» — это не инструкция. Что конкретно проектировать?

Invent Canvas — фреймворк из пяти шагов. Он задаёт рамку: ищи не «фичу с AI», а «что стало возможным благодаря сдвигу». Четыре сдвига из главы 1.1 — это каталог Invent-паттернов.

Пять шагов Invent Canvas

Шаг 1. Какой сдвиг? Какой из четырёх сдвигов открывает возможность, которой раньше не было? Не «как AI ускорит процесс» (это Deploy), а «что становится возможным впервые».

Шаг 2. Кто клиент внутри системы? Как клиент будет генерировать обучающие данные? Какой feedback loop замыкается? Помните: клиент — элемент production pipeline, а не просто потребитель (C2). Если клиент не внутри системы — это Reshape, не Invent.

Шаг 3. Что меняется в поведении клиента? Как мгновенность, персонализация или сервисность изменят решения клиента? Не «клиенту будет удобнее» — а какие новые паттерны поведения возникнут? Instant credit создаёт импульсные покупки. Персональный финансовый агент создаёт привычку ежедневного взаимодействия с банком.

Шаг 4. Где кумулятивный эффект? Через какую шестерёнку Flywheel данные клиента усиливают продукт для всех остальных? Это ключевой вопрос: если нет кумулятивного эффекта — нет конкурентного рва. Любой конкурент повторит продукт за полгода.

Шаг 5. Unit economics per individual. Как считается P&L для одного клиента? Где точка, после которой предельные затраты стремятся к нулю? Если не можете построить модель для одного клиента — вернитесь к шагу 1.

Пример: Invent Canvas для ФинСервис

ФинСервис (средний банк, 30 000 кредитных заявок/мес) проходит Invent Canvas:

Шаг	Ответ
1. Какой сдвиг?	**Один клиент** (C1): персональный финансовый продукт для каждого заёмщика
2. Клиент внутри системы?	Каждое действие заёмщика (заявка, отказ, досрочное погашение, просмотр калькулятора) кормит модель. Feedback loop: предложение → реакция → уточнение модели → лучшее предложение
3. Что меняется в поведении?	Клиент привыкает к мгновенным персональным расчётам. Вместо одного визита за кредитом — 15 взаимодействий за месяц. Банк становится daily habit, а не yearly event
4. Compound effect?	Memory: каждый отказ и каждое принятие обогащают скоринг для всех будущих заёмщиков с похожим профилем. 30 000 заявок/мес = 30 000 обучающих примеров
5. Unit economics?	Предельные затраты предложения: ~$0.02 (API-вызов). Предельный доход: дополнительный кредитный продукт с LTV $2 400. Точка безубыточности: 1 из 120 предложений приводит к сделке

Результат: не «добавим AI к кредитному процессу» (Deploy), не «перестроим андеррайтинг» (Reshape), а персональный финансовый советник для каждого клиента — продукт, которого раньше не существовало.

Когда НЕ использовать Invent Canvas

Invent Canvas не нужен для Deploy и большинства Reshape. Используйте его только когда DRI-оценка показала все три индикатора плюс вы видите новый продукт или

рынок. Если вы пытаетесь натянуть Invent Canvas на обычную автоматизацию — получите переусложнённую стратегию для простой задачи.

Антипаттерн: «Invent — это добавим AI-фичу»

Invent отличается от добавления AI-фичи. Добавить чат-бот к приложению — Deploy. Добавить рекомендации к каталогу — Deploy. Invent — это когда сам продукт невозможен без AI. Когда убираешь AI — продукта не существует, а не «продукт работает медленнее».

Индикатор: если можно описать продукт словами «то же самое, но с AI» — это не Invent.

Три инструмента SELECT — 3-Gate Scoring, DRI и Invent Canvas — отвечают на три последовательных вопроса: какой процесс взять, насколько глубоко менять, и если Invent — что именно создавать. После SELECT вы точно знаете, куда направить усилия.

Но прежде чем переходить к Flywheel — четыре провокационных тезиса, которые стоит обсудить до начала работы.

→ Следующая глава: «*Четыре провокации*»

Четыре провокации

Эта глава — самая неудобная в книге. Четыре тезиса, которые вы не найдёте в отчётах McKinsey и не услышите от интеграторов. Мы считаем, что честная книга полезнее приятной.

Провокация 1. Frozen Middle — не враг

Средний менеджмент — главный барьер AI-трансформации в более чем 60% организаций. Это факт. Но из него обычно делают неправильный вывод: «нужно продавить», «убедить или заменить».

Сопротивление среднего менеджмента — рациональное. Менеджер 15 лет строил свою ценность на трёх вещах: информационная монополия (только он знает, что происходит), ручное распределение работы (только он решает, кто что делает), контроль через присутствие (он управляет, потому что видит процесс). AI уничтожает все три.

Человек, чья ценность обнулилась за полтора года — не саботажник. Он рационально защищает свою позицию. Единственное, что он делает неправильно — защищает старую ценность вместо того, чтобы строить новую.

Решение — перепроектирование роли. Менеджер-контролёр становится менеджером-архитектором процесса. Менеджер-информатор становится аналитиком метрик. Менеджер-распределитель проектирует правила распределения вместо того, чтобы распределять руками.

Воевать с Frozen Middle — значит терять доменных экспертов, чьи знания критичны для Knowledge Base, evals и Data Flywheel. Терять организационную память. Это стратегически глупо.

→ Подробнее: глава 3.1 «Frozen Middle: 6 рычагов»

Провокация 2. Метрики опаснее их отсутствия

Все знают: без метрик плохо. Мало кто говорит о другом: неправильные метрики хуже, чем никаких, потому что создают ложную уверенность.

Кейс Klarna — идеальная иллюстрация. В 2024 году компания отчиталась о результатах AI в поддержке: 2,3 миллиона разговоров в месяц, экономия $40 миллионов, все дашборды зелёные. А потом Klarna вернула людей.

Транзакционные метрики (скорость, стоимость, объём) скрывали проблему в хвосте распределения. Premium-клиенты, сложные ситуации, нетипичные жалобы — AI обрабатывал их формально правильно, но без эмпатии и контекста. Brand damage копилось 12 месяцев, пока кто-то не посмотрел на метрику, которую не мерили: brand perception в premium-сегменте. Средняя метрика скрывала провал в хвосте.

Урок: к каждой основной метрике нужна метрика-противовес (*counterbalance*). Мерите cost-per-outcome — следите за quality в тяжёлых случаях. Мерите cycle time — следите за error rate. Снижаете HIR — следите за customer satisfaction в тех кейсах, где человек не вмешался.

Метрика без counterbalance — это приборная панель, на которой не работает часть датчиков. Летите быстро, видите только хорошее.

→ Подробнее: глава 2.5 «Metrics»

Провокация 3. 30–40% позиций среднего менеджмента исчезнут

Самый неудобный тезис, и ни один корпоративный плейбук его не проговаривает напрямую.

При полном Reshape (когда более 50% ключевых процессов перестроены) организация теряет 30–40% позиций среднего менеджмента. Не потому что «сократили», а потому что функция исчезла. Observability заменяет менеджера-информатора. Workflow engine заменяет координатора. Автоматические evals заменяют контролёра. Если три ключевые функции автоматизированы, значительная часть позиций становится redundant.

Что с этим делать? Не замалчивать. Замалчивание превращает Frozen Middle в Revolting Middle — люди чувствуют угрозу, не слышат честного разговора и начинают активно саботировать.

Правильный подход — честная сегментация ещё до старта. 60–70% менеджеров переходят на A2 за 12–18 месяцев. 15–20% дорастают до A3. Остальные — зона переквалификации или выхода. Компании, которые проводят этот разговор вовремя и вкладываются в содействие трудоустройству, теряют меньше доменных знаний и проходят трансформацию быстрее. Компании, которые молчат, теряют лучших первыми — они уходят сами.

Провокация 4. Decision Capture окупается даже без AI

Decision Capture — структурированный лог решений — многие воспринимают как «инфраструктуру для AI». Зачем записывать каждое решение, если агент ещё не

запущен?

Потому что лог решений ценен сам по себе. Три немедленных выгоды: аудиторский след (регулятор спрашивает «почему?» — вы показываете лог, а не ищете человека, который помнит), база для обучения (новый специалист поддержки видит 500 реальных решений с ходом рассуждений), аналитика (впервые видны паттерны — где больше ошибок, где процесс тормозит).

Но главный бонус — стратегический. «СервисПро» с годом логов решений запускает AI-агента за 1–2 недели: лог становится готовым набором тестов качества, базой знаний и набором примеров для обучения агента. Компания без логов — за 3 месяца, потому что нужно собирать данные с нуля. Год логов — это фора в 10 недель.

Поэтому рекомендация: начните Decision Capture до того, как выберете первый процесс для AI. Самая дешёвая инвестиция с самым высоким ROI.

→ Подробнее: глава 2.3 «Memory»

Пять принципов: итог Части 1

В этом разделе мы познакомились с пятью принципами, на которых стоит вся методика. Каждый — действие, а не декларация.

1. Перестрой, не автоматизируй (глава 1.2). Не ускоряй старый процесс — спроектируй новый. Правильный вопрос: что становится возможным впервые?
2. Сначала тесты, потом система (глава 1.3). Прежде чем строить AI-агента, определите, как вы поймёте, что он работает. 20 эталонных кейсов, метрика качества, порог отката.
3. Каждый клиент учит систему (глава 1.3). Каждое действие клиента — автоматический обучающий сигнал. Без записи нет памяти, без памяти маховик стоит.
4. Скорость решений, а не скорость работы (глава 1.4). Главная единица оптимизации — *decision latency*: как быстро процесс принимает правильное решение.
5. Регулируй обороты (глава 1.4). Автономия AI растёт через уровни (A1–A5), с границами (*Rulebook*) и безопасным переходом. Каждая метрика имеет противовес.

Что дальше

Если вы CEO — этой части может быть достаточно для принятия решения. Дальше делегируйте.

Если вы руководите трансформацией — переходите к Части 2. Пять глав по шестерёнкам Flywheel: концепт, практики, роли, артефакты, failure modes. Всё что нужно, чтобы запустить маховик.

→ *Часть 2: Flywheel* →

Часть 2. FLYWHEEL

Два цикла, пять шестерёнок, тринадцать практик

Как устроен Flywheel

В Части 1 мы увидели маховик с высоты: два цикла, семь глаголов, общая ось данных. Теперь спускаемся внутрь.

Часть 2 — это пять глав, по одной на каждую шестерёнку Flywheel. Прежде чем нырять в каждую, давайте зафиксируем, как они связаны друг с другом и почему порядок имеет значение.

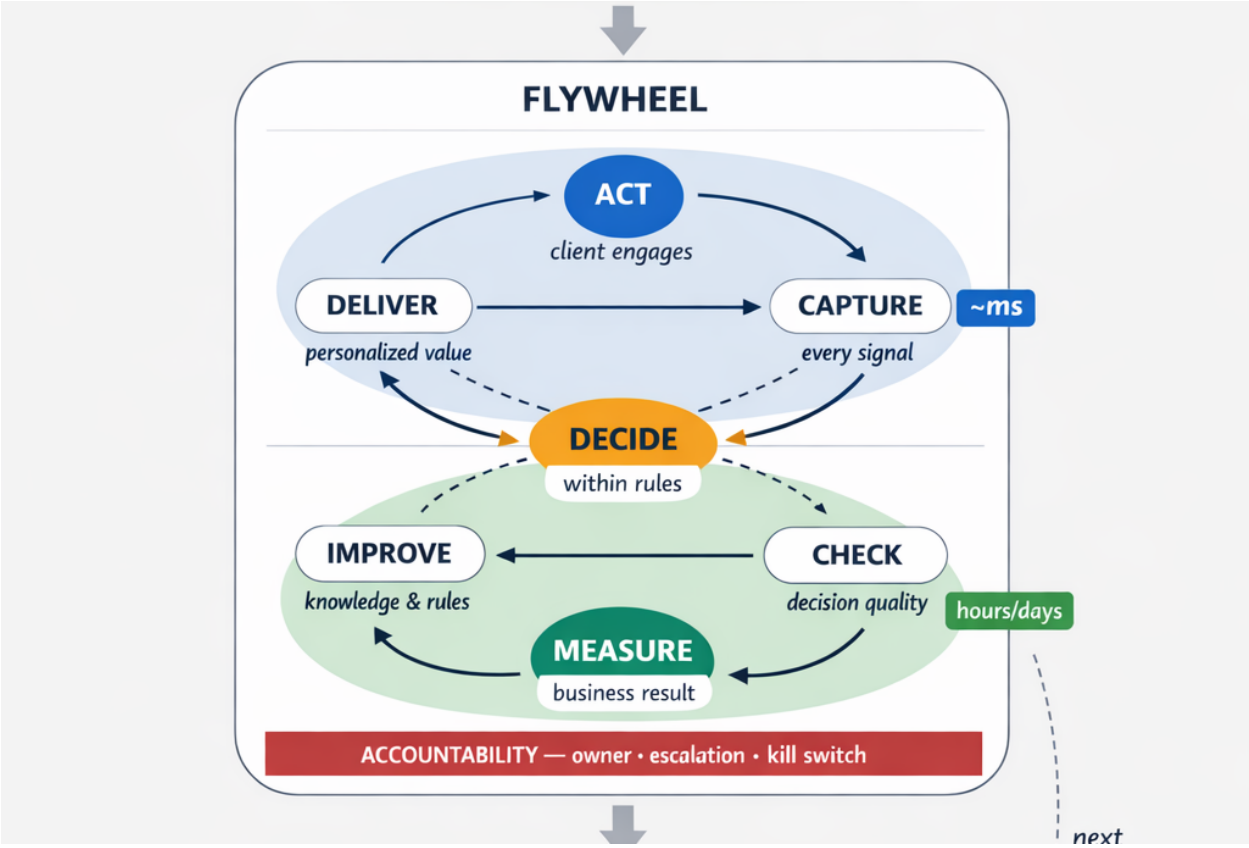

Рис. 2: Flywheel: два цикла, семь глаголов

Два цикла, одна ось

Flywheel — это не один круг с пятью шагами. Это два сцепленных цикла, которые вращаются с разной скоростью и управляются разными рычагами.

Внешний цикл: клиент учит систему. Три шага: DELIVER → ACT → CAPTURE. Система доставляет персонализированную ценность. Клиент действует — покупает, отказывается, задаёт вопрос, молчит. Система захватывает каждый сигнал. Скорость — миллисекунды на каждый контакт. Рычаг управления — продукт и клиентский опыт.

Внутренний цикл: процесс учится сам. Четыре шага: CAPTURE → CHECK → MEASURE → IMPROVE. Захваченные данные проверяются (evals, поиск регрессий). Результат измеряется в бизнес-метриках. Инсайты улучшают знания и правила. Скорость — часы, дни. Рычаг управления — evals, метрики, правила.

DECIDE — перекрёстье. В точке пересечения двух циклов AI принимает решения: улучшенные знания (из внутреннего цикла) превращаются в персонализированные действия (для внешнего цикла). DECIDE работает в рамках правил, которые задал человек-архитектор — уровень автономии, guardrails, red lines.

Почему два цикла, а не один? Потому что у них разная скорость и разные владельцы. Внешний цикл — это про клиента: что он получает, как реагирует, что генерирует. Внутренний — это про качество: работает ли, стало ли лучше, не ломаем ли одно, оптимизируя другое. Когда оба цикла крутятся — получается *кумулятивный эффект*: чем больше клиентов, тем быстрее внешний цикл, тем больше данных для внутреннего, тем точнее следующий DELIVER.

Маппинг: глаголы → шестерёнки → главы

Семь глаголов Flywheel опираются на пять шестерёнок. Каждая шестерёнка — отдельная глава в Части 2.

Глагол	Что происходит	Шестерёнка	Глава
DELIVER	Система доставляет персонализированную ценность	Knowledge Base + Autonomy	2.1 + 2.2
ACT	Клиент действует	*(клиент)*	—
CAPTURE	Каждый сигнал записывается	Memory	2.3
DECIDE	AI принимает решение в рамках правил	Autonomy	2.2
CHECK	Качество проверяется (evals, drift)	Quality Gate	2.4
MEASURE	Бизнес-результат измеряется	Metrics	2.5
IMPROVE	Знания и правила обновляются	Knowledge Base	2.1

ACT — единственный шаг без «нашей» шестерёнки. Его делает клиент. Но именно этот шаг генерирует данные, которые питают оба цикла. Без ACT — нет входных данных. Без входных данных — маховик стоит.

Почему каждый оборот раскручивает следующий

Мы называем это Flywheel — маховик — потому что у системы есть свойство маховика: каждый оборот делает следующий точнее, быстрее и дешевле. Вот как это работает.

Больше клиентов → больше действий (ACT) → больше сигналов в Memory → точнее Knowledge Base → лучше DELIVER → выше доверие клиентов → больше действий → ещё точнее. Это петля положительной обратной связи: выход одного оборота становится усиленным входом следующего.

На первом обороте агент знает мало: база знаний собрана вручную, примеров двадцать, каждое второе решение проверяет человек. На сотом — база обогащена тысячами реальных прогонов, eval-датасет вырос в десять раз, агент работает на A3 и закрывает 70% обращений сам. На тысячном — система знает граничные случаи, которые не описаны ни в одной инструкции, потому что они пришли из реальной практики.

Компания с 10 000 клиентов — это 10 000 параллельных потоков обучения во внешнем цикле. Каждый день. Автоматически.

Конкурент может скопировать вашу архитектуру, нанять тех же интеграторов, купить те же модели. Но он не может скопировать 50 000 размеченных прогонов с обратной связью, которые ваш маховик накопил за полгода работы. Это эффект сложного процента на данных — и это главное конкурентное преимущество, которое создаёт процесс-как-продукт. (*Прим.: “Data flywheel” — маховик, раскручивающийся через данные).

Что ломается без каждой шестерёнки

Прежде чем разбирать шестерёнки по одной — короткий тест. Что будет, если убрать любую из них?

Без Knowledge Base — AI галлюцинирует. Нет контекста, нет правил домена, каждый ответ — угадайка.

Без Autonomy — AI либо слишком свободен (принимает решения, на которые не имеет права), либо бесполезен (ждёт подтверждения на каждый чих). Нет чётких границ — нет доверия.

Без Memory — оба цикла не крутятся. Каждый прогон начинается с нуля. Нет данных для CHECK, нечего MEASURE, нечем IMPROVE.

Без Quality Gate — деградация невидима. Модель дрейфует, промпт сломался, accuracy упала на 15% — а никто не заметил. Через месяц — «AI не работает, мы же говорили».

Без Metrics — Process Owner не может управлять. Бизнес не видит ROI. Оптимизация вслепую: улучшаем скорость, ломаем качество.

Пять шестерёнок — это минимальная комплектация. Уберите любую — и маховик

останавливается.

Два кейса в Части 2. Основная нить — «СервисПро» (обработка обращений): на этом кейсе мы покажем все пять шестерёнок и эволюцию процесса от первого месяца до масштабирования. «ФинСервис» (кредитные заявки) появляется в секциях «Клиентский контур» — там, где важно показать, как маховик работает на стороне клиента: персонализация, обучение на каждом контакте, скорость адаптации.

Готовы? Начинаем с первой шестерёнки — Knowledge Base: откуда AI знает, как работать.

→ Следующая глава: «*Knowledge Base — откуда AI знает, как работать*»

Knowledge Base: откуда AI знает, как работать

В «СервисПро» три старших специалиста поддержки. Олег работает 12 лет в enterprise IT и знает, что обращения от клиентов с кастомной интеграцией — повышенный риск, даже если симптомы выглядят типово. Марина помнит, что после обновления продукта всплывают обращения по старым API — это не баг, а нормальная волна. Игорь знает десять классов проблем, которые требуют специальной обработки: интеграции, security incidents, мультипродуктовые кейсы и другие.

Теперь представьте: AI-агент обрабатывает обращения клиентов. Откуда он возьмёт эти 12 лет контекста о product quirks и client-specific configurations? Из промпта на три абзаца? Из общих знаний модели об IT? Нет. Без формализованного знания домена агент будет уверенно и быстро принимать плохие решения. Это ошибка архитектуры.

Knowledge Base — первая шестерёнка Data Flywheel. Всё начинается здесь.

Зачем эта шестерёнка

Без Knowledge Base агент работает «вслепую» — на основе общих знаний модели, без контекста вашей отрасли, ваших продуктов, ваших исключений. Результат предсказуем: агент закрывает обращение по Template, которое старший специалист отклонил бы за 30 секунд. Или эскалирует Enterprise-клиента как стандартное обращение, потому что не знает, что для этой категории другие SLA и процедуры.

Главная проблема — *уверенная некомпетентность*. Агент без KB не говорит «я не знаю». Он выдаёт правдоподобный, но неверный ответ с высокой оценкой уверенности. Чем лучше модель — тем убедительнее ошибка.

Этот риск можно предотвратить. Когда агент не уверен или попадает вне пределов KB — нужна красная линия: прямая эскалация к человеку, без риска неправильного ответа. Это задача второй шестерёнки (Autonomy).

Второй эффект менее очевиден: без KB каждый прогон маховика начинается с нуля. Агент не учится на предыдущих решениях, потому что ему некуда записать и неоткуда прочитать накопленный опыт. Memory (глава 2.3) без KB — как дневник, который никто не перечитывает.

Три слоя Knowledge Base

KB — это три слоя, каждый со своей природой и частотой обновления.

Слой 1: факты домена

То, что верно для вашей отрасли и вашей компании. Типы продуктов, SLA-уровни по категориям клиентов, критичность проблем (P1–P4), интеграционные спецификации, requirements по compliance, карта типов проблем. Этот слой меняется редко — раз в квартал или при выходе нового продукта.

В «СервисПро» это выглядит так: таблица типов клиентов (стартап, mid-market, Enterprise), для каждого — пороговые значения SLA (response time, resolution time), критичность проблем (P1 = production down требует immediate response), обязательные проверки (security, compliance), структура интеграций. Раньше это «знал Олег». Теперь это структурированный файл, который видит и агент, и новый специалист поддержки.

Слой 2: политики и правила

Как принимать решения в типовых ситуациях. Дерево решений для стандартных обращений, правила эскалации, красные линии (что запрещено при любых обстоятельствах), порядок действий при сложных случаях. Этот слой меняется чаще — каждый месяц появляются уточнения, новые типы интеграций, обновлённые требования по compliance.

Пример из «СервисПро»: «Критичность P1 (production down) всегда немедленная эскалация, вне зависимости от типа клиента» — красная линия. «При попытке получить конфиденциальные данные (credentials, API keys) — стоп + security team» — красная линия. «SLA breach risk (время до deadline < 20% оставшегося SLA) — автоматическая эскалация старшему специалисту» — правило. «Клиент с кастомной интеграцией и мультипродуктовая проблема — проверить interdependencies по списку» — политика.

Слой 3: примеры решений

Реальные кейсы с объяснением логики. Решённое обращение от клиента с кастомной интеграцией — как агент определил типичную для этого интеграции проблему и предложил решение. Эскалированное обращение с попыткой получить конфиденциальные данные — какие красные флаги увидел агент. Этот слой растёт непрерывно и является самым ценным: через полгода работы маховика здесь накопятся сотни размеченных примеров, которые невозможно купить или скопировать.

Связь между слоями:

Слой	Что содержит	Частота обновлений	Кто обновляет
Факты домена	SLA, типы продуктов, интеграции, критичность	Раз в квартал	Доменный эксперт
Политики и правила	Деревья решений, красные линии, эскалации	Ежемесячно	Доменный эксперт + PO
Примеры решений	Реальные кейсы с reasoning и результатом	Непрерывно	Из Decision Log (автоматически)

Третий слой — мост к шестерёнке Memory (глава 2.3). Когда маховик крутится, лог решений автоматически подпитывает KB новыми примерами. Но для старта нужен «посевной фонд» — 20–50 размеченных примеров, собранных вручную.

Как строить KB: итерация, не водопад

Самая частая ошибка — пытаться описать «всё» перед запуском. Команда тратит три месяца на документирование процесса, получает 80-страничную инструкцию, которая устаревает к моменту завершения. Это болото данных (*data swamp*): объём есть, пользы нет.

KB строится итеративно, в три шага.

Шаг 1: сбор «скелета» (неделя 1–2). Посадите доменного эксперта и владельца процесса (PO) за стол. Задача — описать не весь процесс, а только типовой путь (*happy path*): что происходит с обращением, которое решается стандартно, без исключений. Для поддержки это 4–6 шагов: приём обращения → классификация типа проблемы → поиск в KB → подготовка решения → отправка. На каждом шаге — какие данные нужны и какое решение принимается. Результат: 2–3 страницы, не больше.

Шаг 2: добавление исключений (неделя 3–4). Теперь спрашиваем эксперта: «Что может пойти не так?» Не абстрактно, а по каждому шагу скелета. На шаге классификации: «Какие обращения выглядят типово, но скрывают кастомную интеграцию или security issue?» Именно здесь всплывает неявное знание — то, что Олег знает 12 лет, но никогда не формализовал. Записываем как правила и примеры. Результат: KB вырастает до 8–15 страниц.

Шаг 3: калибровка по живому потоку (неделя 5+). Запускаем агент на уровне A1 (подсказки специалисту). Каждый раз, когда специалист не согласен с подсказкой — это сигнал о пробеле в KB. Специалист отмечает: «Агент предложил типовое решение, но я вижу, что это клиент с кастомной интеграцией — нужно проверить interdependencies.»

Этот сигнал превращается в новое правило или пример. KB растёт органически, по реальным пробелам, а не по фантазиям о том, что «может пригодиться».

Через месяц работы на A1 ваша KB будет точнее, чем любой документ, написанный заранее. Потому что она откалибрована на реальном потоке заявок, а не на воспоминаниях эксперта о типичных случаях.

Пример: Knowledge Base процесса обработки обращений «СервисПро»

Мария (PO процесса поддержки) начала с Олегом за круглым столом. Олег за два дня описал типовой путь обращения и 15 ключевых правил, которые раньше жили только в его голове.

Первая версия KB:

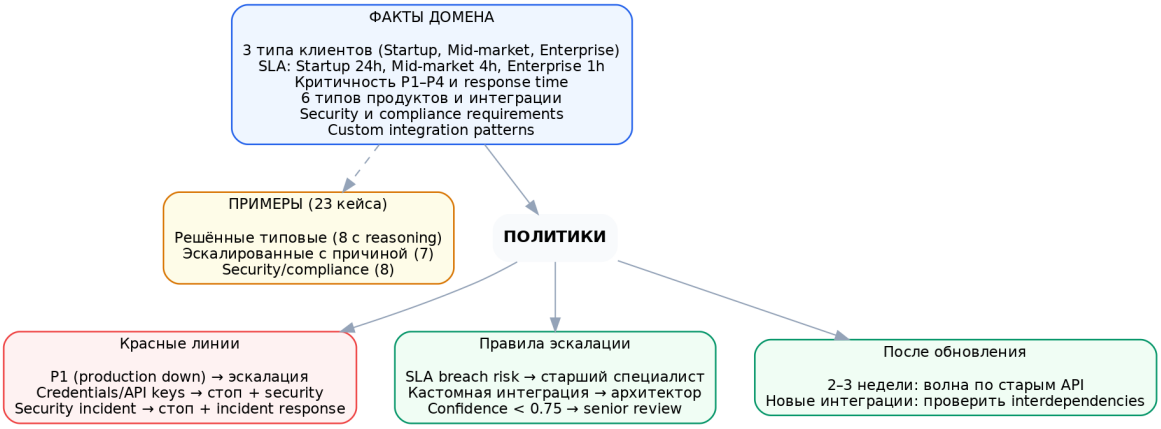

Рис. 3: Структура Knowledge Base «СервисПро»

Через месяц работы на A1 KB выросла с 23 до 89 примеров — каждый добавлен потому, что агент ошибся или специалист не согласился с подсказкой. К четвёртому месяцу — 340 примеров, и агент перешёл на A2 (готовит решение, специалист подтверждает).

Важная деталь: рост KB замедляется. Первые 50 примеров закрывают 80% ситуаций. Следующие 200 — оставшиеся 15%. Последние 5% — экзотика, которая всегда будет уходить на человека. Это нормально: KB не должна покрывать 100%.

Клиентский контур: KB на стыке с клиентом

Три слоя KB, описанные выше, — это знание о домене. Но есть ещё один слой, который принципиально меняет качество DELIVER во внешнем цикле Flywheel: клиентский контекст.

Клиентский контекст — это персональный слой KB: история взаимодействий конкретного клиента, его предпочтения, предыдущие обращения, паттерн поведения, предпочтительный канал. Чем богаче контекст — тем точнее персонализация (сдвиг C1: один клиент).

Пример на «ФинСервис» (обработка кредитных заявок). KB знает общие правила скоринга. Но клиентский контекст знает, что заёмщик Алексей уже брал автокредит, досрочно погасил его, отклонил страховку, предпочитает мобильный канал и заходит в приложение вечером. Это знание о конкретном клиенте, которое превращает «стандартное предложение для сегмента» в «персональное предложение для Алексея».

Клиентский контекст работает на два цикла. Во внешнем: обогащает DELIVER — агент формирует ответ с учётом истории клиента. Во внутреннем: данные из клиентского контекста попадают в Memory через CAPTURE и обогащают модели для всех клиентов с похожим профилем.

Практическое правило: если ваш процесс клиентский (поддержка, продажи, кредитование) — выделите клиентский контекст как отдельный слой KB. Если процесс внутренний (отчётность, HR) — этот слой не нужен.

Роль: доменный эксперт

Knowledge Base пишет тот, кто знает домен — разработчик или аналитик данных здесь не помогут. KB пишет тот, кто знает домен: старший специалист поддержки, ведущий архитектор, руководитель направления поддержки — тот самый человек, в чьей голове живут 12 лет опыта в enterprise IT.

Доменный эксперт (*Domain Expert*) — одна из трёх ключевых ролей методики. Его задача:

- Формализовать неявное знание в правила и примеры
- Валидировать решения агента на сложных случаях
- Обновлять KB при изменении домена (новые продукты, обновления платформы, новые типы интеграций)
- Разбирать ошибки агента и превращать их в новые записи KB

Это не полная занятость. В «СервисПро» Олег тратит 4–6 часов в неделю на работу с KB: 2 часа — разбор расхождений между агентом и специалистами, 2 часа — обновление правил, 1–2 часа — калибровочные сессии с Марией (PO).

Частая проблема: доменный эксперт не хочет делиться знаниями. Его власть строилась на информационной монополии — «только я знаю, как это работает». Формализация в KB уничтожает эту монополию. Это не каприз, а рациональная реакция

(подробнее — в главе 3.1 про Frozen Middle). Решение: переопределить роль эксперта с «человека, который знает» на «человека, который учит систему». Это повышение, не понижение.

Артефакт: Knowledge Base

Формализованные знания домена в структурированном формате, доступные агенту и команде. Три слоя (факты, политики, примеры), версионирование, лог изменений.

Формат не принципиален: это может быть набор файлов, база в Notion, структурированный JSON — главное, чтобы агент мог читать, а человек — редактировать. Избегайте 200-страничных Word-документов: агент не понимает контекст длинного текста так же хорошо, как структурированные правила.

Когда KB не нужна

Не каждый процесс требует полноценной Knowledge Base.

Не нужна, если процесс полностью определяется формальными правилами без исключений. Расчёт налога по фиксированной формуле, конвертация валюты по курсу ЦБ, маршрутизация заявки по единственному критерию. Здесь достаточно кода, не KB.

Не нужна, если процесс проходит меньше 20 прогонов в месяц. Затраты на формализацию не окупятся — проще держать эксперта на ручном режиме.

Нужна облегчённая версия (только слой 1 — факты домена), если процесс на уровне A1 (подсказки) и не планируется повышение автономии в ближайшие 6 месяцев.

Главная ошибка: «мёртвая» KB

Knowledge Base, которая не обновлялась 3 месяца — мертва. Домен изменился: новые продукты, обновления платформы, новые типы интеграций. Агент работает по устаревшим правилам и принимает решения, которые были верны в прошлом квартале, но не сегодня.

В «СервисПро» это едва не случилось: вышло обновление продукта v4.2 с new API, а KB продолжала работать по старым endpoint'ам. Две недели агент направлял клиентов на устаревшие интеграции. Поймали, когда Олег заметил аномалию в пакетном ревью — 12 обращений с одинаковой ошибкой, которые указывали на старую документацию в KB.

Решение — встроить обслуживание KB в регулярный ритм. Срок жизни записей

(TTL): факты домена — проверка раз в квартал, политики — раз в месяц, примеры — без TTL, но с пометкой о дате. Если запись старше TTL и не подтверждена — агент видит предупреждение, а PO получает оповещение.

Связь со следующей шестерёнкой

KB даёт агенту контекст — он знает *что* и *как*. Во внешнем цикле KB (вместе с клиентским контекстом) питает DELIVER — персонализированную ценность. Во внутреннем — IMPROVE обогащает KB по результатам метрик и аналитики.

Но знать недостаточно. Сколько свободы агент получает? Может ли он принять решение сам или только предложить вариант? Где проходят границы? Это вопрос DECIDE — перекрёстья двух циклов.

Это вопрос Autonomy — второй шестерёнки маховика.

→ Следующая глава: «*Autonomy: сколько свободы у AI и где границы*»

Autonomy: сколько свободы у AI и где границы

Мария запускает AI-агент для обработки поддержки обращений клиентов «СервисПро». Knowledge Base собрана, эксперты формализовали правила. И тут — вопрос, который вызывает самые жаркие споры в команде: «А что агент имеет право делать?»

Одна группа говорит: «Пусть подсказывает специалисту, а решает человек. Безопасно.» Другая: «Зачем тогда агент? Пусть закрывает сам — мы же собрали для него все правила.» Третья, из комплаенса: «У нас SLA-требования. Агент не может закрывать P1-обращения без подтверждения. Точка.»

Все три группы правы — для разных ситуаций. Проблема в том, что без системы это превращается в политический торг: кто громче крикнет, тот и определит границы.

Autonomy — вторая шестерёнка Data Flywheel. Она отвечает на вопрос: сколько свободы у AI и где проходят линии, которые нельзя пересекать.

Зачем эта шестерёнка

Без определённой автономии возникают две симметричные проблемы.

Слишком мало свободы. Агент подсказывает, специалист решает. Звучит безопасно. На практике — специалист игнорирует подсказки через две недели. У него свой опыт, своя интуиция, и ещё один «советчик» в интерфейсе только мешает. HIR (доля вмешательства человека) остаётся на 95%, маховик не крутится. Организация заплатила за API, но ничего не изменилось.

Слишком много свободы. Агент решает сам, без ограничений. Пока всё идёт хорошо — красивые метрики. Но когда агент закроет обращение с попыткой раскрыть конфиденциальные данные или пропустит P1-инцидент без эскалации — последствия необратимы. Причём узнаете вы об этом не в момент ошибки, а через день, когда клиент напомнит о SLA-breach или security team обнаружит issue.

Autonomy — это не переключатель «вкл/выкл». Это шкала с пятью уровнями, ограждениями на каждом уровне и красными линиями, которые нельзя пересекать ни при каких обстоятельствах. В главе 1.3 мы ввели таблицу A1–A5. Здесь разберём, как ей пользоваться.

Практика 1: Лестница автономии A1–A5

Проблема

Команда спорит об уровне автономии, но у спора нет общего языка. «Давайте дадим агенту побольше свободы» — это A3 или A5? «Пусть человек проверяет» — каждое решение или пакетно? Без шкалы разговор буксует в абстракциях.

Инструмент

Пять уровней — от «агент подсказывает» до «агент действует полностью автономно». Каждый уровень определяет, что делает AI и что делает человек. Переход между уровнями — по данным, а не по ощущениям.

Уровень	Что делает AI	Что делает человек	Когда применять
A1 — Ассистент	Предлагает варианты	Принимает решение сам	Старт: нет данных о качестве агента
A2 — Советник	Готовит решение	Проверяет и утверждает каждое	Есть первые данные, но доверие не построено
A3 — Со-пилот	Закрывает обращение	Утверждает пакетно, разбирает исключения	Данные показывают accuracy выше порога
A4 — Автопилот с надзором	Действует сам в штатных ситуациях	Мониторит метрики, вмешивается при аномалиях	Стабильное качество на большом объёме
A5 — Полная автономия	Действует сам, включая нестандартные	Проектирует правила и границы	Низкий риск, высокая обратимость

Ключевое: уровень назначается не процессу целиком, а *типу решений внутри процесса.* В процессе обработки обращений «СервисПро» стандартные обращения — на A3, сложные — на A2, с security риском — на A1 (красная линия, всегда человек). Один процесс, три уровня одновременно.

Как определить стартовый уровень

Два фактора: обратимость решения и доступность данных для оценки качества.

	Данные есть (можно оценить качество за 30 дней)	Данных нет
Решение обратимо (можно отменить без последствий)	Старт с A2, быстрый переход на A3	Старт с A1

	Данные есть (можно оценить качество за 30 дней)	Данных нет
Решение необратимо (отмена дорога или невозможна)	Старт с A2, медленный переход на A3 с Dual Run	Старт с A1, длительная калибровка

Критерии перехода между уровнями

Переход — решение на основе данных. Три критерия, все обязательны:

Критерий 1: accuracy. Оценка качества агента (*eval score*) на тестовом наборе выше порога. Порог устанавливает PO: для обработки обращений «СервисПро» — 90% на стандартных обращениях, 85% на сложных. Для генерации обычных ответов порог может быть 70% — цена ошибки ниже.

Критерий 2: стабильность. Eval score не падает последние 4 недели. Разовый всплеск качества после обновления промпта — не основание для перехода. Нужна стабильная серия.

Критерий 3: покрытие исключений. Агент корректно обрабатывает минимум 80% известных сложных ситуаций из тестового набора. Если агент отлично работает на типовых обращениях, но «ломается» на security issues или Enterprise-клиентах — переход на следующий уровень только для типовых, не для всех.

Пример: лестница «СервисПро»

Месяц 1 (A1). Агент видит обращение и показывает специалисту рекомендацию: «Типовая конфигурация API, проверить документацию v4.1, оценка уверенности 0.87. Основание: обращение соответствует паттерну K15, клиент Premium SLA.» Специалист принимает решение сам. За месяц — 800 обращений в день, агент совпал со специалистом в 89% случаев. В 7% агент был неправ (неполная KB). В 4% — специалист был неправ (агент учёл данные, которые специалист пропустил).

Месяц 2–3 (A2). KB дополнена по результатам расхождений. Агент теперь готовит полное решение: рекомендацию по решению, ссылки на KB, steps для клиента, с обоснованием. Специалист утверждает каждое. Eval score на стандартных обращениях: 94%. Время специалиста на обращение: с 4 минут до 50 секунд (проверка вместо анализа).

Месяц 4 (A3 для стандартных). Данные показали: accuracy 96% на стандартных обращениях стабильно 4 недели. PO Мария принимает решение: стандартные обращения переходят на A3. Агент закрывает, специалист проверяет пакетно — 100 решений утром, 100 вечером. Сложные обращения остаются на A2. Красные линии

(security incidents, P1 без подтверждения) — на A1.

Месяц 8 (A4 для стандартных). Eval score на стандартных: 97% стабильно 3 месяца. 150 000 решённых обращений в логе. Мария переводит стандартные на A4: агент закрывает, специалист не проверяет каждое решение, а мониторит дашборд — HIR, eval score, reopening rate аномалии. Вмешивается только при отклонении от нормы.

Антипаттерн: «Давайте сразу A5»

Руководство видит впечатляющие demo и хочет «полную автоматизацию» за квартал. Проблема: A5 без прохождения A1–A4 — как строить пятый этаж без фундамента. Нет данных для eval, нет калиброванной KB, нет понимания failure modes. Результат: красивый запуск, тихая катастрофа через 2–3 месяца.

Лестница — не бюрократия. Это инвестиция в данные. Каждый уровень производит данные, необходимые для следующего. A1 производит baseline: как часто агент совпадает с человеком. A2 производит калибровку: где именно агент ошибается. A3 производит масштаб: достаточно данных для статистической значимости. Пропуск уровня — пропуск данных.

Практика 2: AI Rulebook

Проблема

У вас есть уровни автономии. Агент знает, что на A3 он может принимать решения по стандартным заявкам. Но *какие именно* ограничения действуют на каждом уровне? «Не одобряй мошенничество» — понятно. А «не отклоняй заявку только на основании отрасли» — это ограничение или рекомендация? «Эскалируй, если не уверен» — при какой оценке уверенности?

Без формализованных правил агент работает в серой зоне. А серая зона в бизнес-процессе — это риск, который никто не измерил.

Инструмент: три слоя Rulebook

Rulebook — документ, который определяет границы поведения агента. Три слоя, от жёстких к гибким.

Слой 1: красные линии (*Red Lines*). Запреты, которые действуют всегда, на любом уровне автономии, без исключений. Пересечение красной линии — инцидент, который требует разбора.

Примеры для процесса поддержки: - Security incidents (попытка получить credentials, anomalies) — всегда на человека - P1-обращения (production down) — всегда немедленная эскалация - Попытка раскрыть конфиденциальные данные — стоп + security team - Агент никогда не предоставляет доступ или закрытые данные клиенту

Красных линий не должно быть много. 5–10 для одного процесса — достаточно. 50 красных линий парализуют агента.

Слой 2: ограждения (*Guardrails*). Правила, которые ограничивают свободу агента в рамках уровня. В отличие от красных линий, ограждения могут меняться при переходе на следующий уровень.

Примеры: - На A2: агент не может закрыть обращение без указания конкретного решения из KB - На A3: агент эскалирует при оценке уверенности ниже 0.75 - На A4: агент уведомляет PO, если distribution решений за день отличается от средней за месяц более чем на 2 стандартных отклонения

Слой 3: плейбук (*Playbook*). Рекомендации и лучшие практики. Не обязательные к выполнению, но полезные. Обновляются чаще всего — по мере накопления опыта.

Примеры: - При обращении от Enterprise-клиента — проверить SLA requirements, даже если задача выглядит типово - Если клиент использует custom API integration — проверить interdependencies с другими компонентами - Сезонная корректировка: после релиза новой версии 2–3 недели волна обращений по интеграции

Пример Rulebook «СервисПро»

Живой документ, не мёртвый регламент

Rulebook, который не менялся 3 месяца, — мёртвый Rulebook. Домен меняется: новые требования SLA, новые типы security risks, новые версии продуктов. Правило обслуживания: PO проверяет Rulebook раз в месяц, доменный эксперт — при каждом инциденте.

Каждое изменение версионируется. Rulebook v2.4 — значит, было 23 предыдущих обновления. Если нужно откатить агента на предыдущее поведение — откатываем и Rulebook, и промпт, и KB.

Антипаттерн: 200-страничная политика

Отдел комплаенса пишет документ на 200 страниц, покрывающий все возможные ситуации. Документ идеально точен юридически и полностью бесполезен операционно. Агент не может следовать 200-страничной инструкции — контекстное окно конечно.

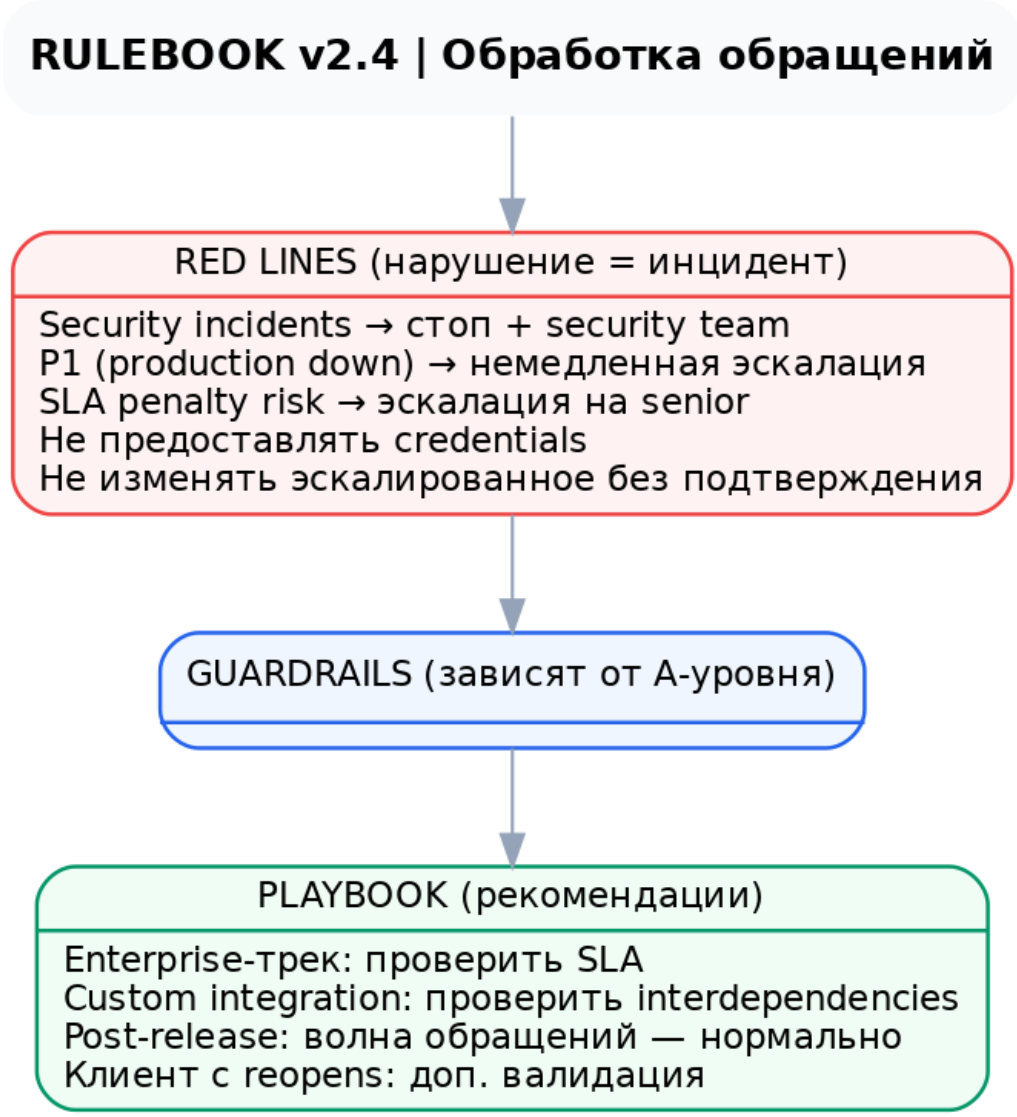

Рис. 4: Rulebook v2.4 «СервисПро»

Команда не может поддерживать 200-страничный документ — обновление занимает месяц согласований.

Rulebook работает, когда он короткий, структурированный и обновляемый. 2–3 страницы для одного процесса. Если длиннее — декомпозируйте на несколько Rulebook по типам решений.

Практика 3: Dual Run

Проблема

Вы на A2, данные хорошие, хотите перейти на A3. Как это сделать безопасно? Переключить всех сразу — рискованно: если агент ошибётся на реальном потоке, последствия неотменяемы. Не переключать — маховик стоит.

Инструмент: четыре режима перехода

Dual Run — протокол безопасного повышения автономии. Четыре режима, каждый следующий увеличивает долю AI.

Shadow (теневой). Агент работает параллельно с человеком, но его решения никуда не идут. Человек не видит подсказок агента — решает как обычно. После решения — сравниваем: что сказал агент, что сказал человек. Это чистый A/B без влияния на процесс. Длительность: 2–4 недели.

Зачем shadow, если агент уже работал на A1? На A1 оператор видит подсказку — это влияет на его решение (*anchoring bias*). Shadow даёт несмещённую оценку: насколько агент совпадает с человеком, когда человек не знает о подсказке.

Canary (канареечный). 5–10% реального потока переводится на новый уровень. Остальные 90–95% — по-старому. Если canary-группа показывает нормальные метрики 2 недели — увеличиваем долю. Длительность: 2–4 недели.

Gradual (постепенный). Доля AI-потока увеличивается по расписанию: 10% → 25% → 50% → 75% → 100%. На каждом шаге — проверка метрик. Если что-то ломается — откат на предыдущий шаг. Длительность: 4–8 недель.

Full (полный). 100% потока на новом уровне. Мониторинг усилен первые 2 недели: ежедневная проверка метрик вместо еженедельной.

Критерии перехода между режимами

Переход	Критерий	Аварийная кнопка
Shadow → Canary	Совпадение с человеком > 90%	Совпадение < 80% две недели подряд
Canary → Gradual	Eval score canary ≥ eval score control	Eval score canary на 5+ пп ниже control
Gradual → Full	Все метрики стабильны 4 недели	Любая метрика вне допустимого коридора

Аварийная кнопка — не повод для паники, а повод для диагностики. Агент не справился с canary? Возможно, KB неполная для конкретного сегмента. Eval score упал на gradual? Возможно, на большем потоке появились кейсы, которых не было в тестовом наборе. Каждый kill-сигнал — информация для улучшения, не для отката проекта.

Пример: Dual Run в «СервисПро»

Мария готовит переход стандартных обращений с A2 на A3.

Неделя 1–2 (Shadow). Агент обрабатывает все обращения параллельно, но специалисты не видят его решений. Результат: совпадение 94%. Расхождения в 6% проанализированы — в 4% агент был прав (специалист пропустил данные), в 2% — не хватало контекста в KB. KB дополнена.

Неделя 3–4 (Canary). 80 обращений в день из 800 переходят на A3: агент закрывает, специалист подтверждает пакетно утром и вечером. Eval score canary: 95%. Eval score control (A2): 94%. Canary лучше — потому что специалист на A2 иногда устаёт и ошибается, а агент стабилен.

Неделя 5–8 (Gradual). 10% → 25% → 50% → 75%. На 50% заметили аномалию: eval score по обращениям с custom API упал до 82%. Причина: при росте потока появились обращения с мало документированными интеграциями — категория, которой не было полностью в KB. Пауза на 50%. Олег (доменный эксперт) добавляет правила для этих интеграций. Через неделю — eval score восстановился до 93%. Продолжаем: 75% → 100%.

Неделя 9+ (Full). 100% стандартных обращений на A3. Ежедневный мониторинг первые 2 недели, затем — еженедельный. HIR по стандартным обращениям: 8% (в основном — случаи с неполной информацией, которые агент корректно эскалирует).

Антипаттерн: «Пилот прошёл — раскатываем на всех»

Пилот на 50 заявках показал отличный результат. Руководство в восторге. Решение: «Раскатываем на весь поток». Через неделю — 15 жалоб от клиентов. Потому что пилот не встретил нетипичные случаи, сезонные аномалии и edge cases, которые появляются только на масштабе.

Dual Run — инструмент, который производит данные. Shadow производит baseline. Canary производит сравнение. Gradual производит стресс-тест на масштабе. Пропуск любого этапа — пропуск данных, которые вам потом не хватит.

Capability и Permission: важное различие

Агент *может* закрыть обращение о security incident — технически ничего не мешает. Но *имеет ли он право*? Способность модели (*capability*) и разрешение (*permission*) — разные вещи. Современные модели способны на многое. Вопрос Autonomy — не «что AI умеет», а «что AI разрешено делать в данном процессе на данном уровне».

Это связь с принципом Human-as-Architect (глава 1.4): человек проектирует систему и задаёт границы. AI исполняет внутри этих границ. Границы определяются Rulebook. Уровень свободы внутри границ определяется L-уровнем. Всё вместе — Autonomy.

Клиентский контур: автономия по отношению к клиенту

A-уровни описывают автономию в целом. Но есть отдельный вопрос: что агент имеет право делать по отношению к клиенту?

Может ли агент инициировать контакт? Предлагать цену? Менять условия договора? Отказывать в услуге? Каждое из этих действий требует отдельного решения об автономии — и отдельных guardrails.

Два ключевых guardrail клиентской автономии:

Trust — доверие клиента к AI. Если клиент не знает, что с ним общается агент, — это одна ситуация. Если знает и согласен — другая. Правило: прозрачность уровня автономии для клиента. В «ФинСервис» агент говорит: «Я подготовил для вас расчёт. Если хотите обсудить с консультантом — нажмите здесь.»

Explainability — объяснимость решений. Клиент, которому отказали в кредите, имеет право знать почему. Агент должен уметь объяснить своё решение на языке клиента, а не показывать confidence score. В «ФинСервис»: «Мы не можем одобрить заявку сейчас, потому что текущая долговая нагрузка превышает 45% дохода. Вот что можно сделать...»

Пример ФинСервис: агент сам предлагает рефинансирование, когда видит, что у клиента закрывается автокредит. Это A3 по клиентской автономии: агент инициирует контакт и предлагает конкретные условия. Но в рамках Rulebook: максимальная сумма предложения ограничена, ставка — по тарифной сетке, red line — агент не может менять условия действующего договора.

Accountability: сквозное ограничение

Accountability не вынесена в отдельный принцип, потому что пронизывает каждый из четырёх. Но именно поэтому её легко потерять — когда «за всё» отвечают все, не отвечает никто.

Правило: ответственность привязана к уровню автономии. Чем выше автономия — тем выше уровень владельца и жёстче механизм остановки.

Уровень	Владелец ответственности	Эскалация	Аварийная кнопка
A1–A2 (Copilot)	Пользователь	Team lead	Пользователь отменяет рекомендацию
A3 (Human-in-the-loop)	Process Owner	VP / Head of function	PO останавливает агента
A4–A5 (автономный агент)	PO AI-процесса	C-suite	Автоматический предохранитель + ручной override

Аварийная кнопка — механизм немедленной остановки агента. На A1–A2 это просто: пользователь игнорирует подсказку. На A4–A5 нужен автоматический предохранитель: если eval score падает ниже порога, или HIR аномально низкий (агент перестал эскалировать), или количество жалоб превысило норму — агент автоматически переходит на A2 до разбора. Плюс ручной override: PO может остановить агента одной кнопкой.

Цепочка ответственности определяется до запуска, а не после инцидента. Если перед запуском нельзя ответить «кто владелец, как эскалация, где kill switch» — процесс не готов к production.

Роль: нет отдельной роли

Autonomy не имеет выделенной роли — она управляется совместно. PO определяет целевой L-уровень и критерии перехода. Доменный эксперт формулирует красные линии и правила. Комплаенс утверждает Rulebook. Решение о переходе на следующий уровень — за PO, на основе данных.

Артефакт: Rulebook

Документ из трёх слоёв (Red Lines → Guardrails → Playbook) для одного процесса. Версионируется, обновляется ежемесячно, откатывается вместе с промптом и KB при

деградации.

Когда Autonomy можно упростить

Процесс на A1 без планов повышения. Если агент навсегда останется подсказчиком (copilot) — Rulebook избыточен. Достаточно красных линий в промпте.

Процесс с одним типом решений. Нет нужды в дифференциации по типам — один A-уровень на весь процесс, упрощённый Rulebook.

Внутренний процесс без регуляторных требований. Генерация внутренних отчётов, резюмирование встреч — риск низкий, Dual Run можно сократить до Shadow + Full.

Главная ошибка: размывание границ автономии

Агент стартует на A2 с чёткими ограничениями. Через 3 месяца никто не помнит, какие ограничения действуют. Новый оператор думает, что агент на A3. PO не проверял Rulebook два месяца. Фактический уровень автономии незаметно превышает формальный — без данных, без eval, без осознанного решения. Это «размывание границ» (*autonomy creep*) — опасное состояние, когда система работает вне предусмотренных пределов.

Решение: Autonomy Register — одностраничная таблица на каждый процесс.

Процесс	Тип решений	Текущий уровень	Дата последнего перехода	Целевой уровень	Следующий ревью
Обращения клиентов	Стандартные	A4	01.11.2025	A4	01.02.2026
Обращения клиентов	Сложные	A2	15.07.2025	A3	15.01.2026
Обращения клиентов	Security incidents	A1 (красная линия)	—	A1	—

Эта таблица висит на видном месте. PO обновляет её при каждом переходе и ревьюит ежеквартально. Если фактическое поведение расходится с таблицей — инцидент.

Связь со следующей шестерёнкой

Агент работает в рамках Autonomy: принимает решения на своём L-уровне, соблюдает Rulebook. Во внешнем цикле это момент DECIDE — перекрёстье, где

знания превращаются в действия. Во внутреннем — уровень автономии определяет, как именно проходит CHECK и IMPROVE.

Но куда записываются эти решения? Как через месяц понять, было ли решение верным? Как во внешнем цикле превратить ACT клиента в данные для обучения?

Это вопрос Memory — третьей шестерёнки маховика. И точка CAPTURE в обоих циклах.

→ Следующая глава: *«Memory: что решили, почему и оказалось ли верным»*

Memory: что решили, почему и оказалось ли верным

Через четыре месяца работы агент «СервисПро» обработал 80 000 обращений клиентов. Мария открывает дашборд и видит: HIR снижается, eval score растёт, стоимость на решение падает. Всё хорошо. А потом менеджер Enterprise-клиента задаёт вопрос: «Почему процент reopens вырос на 12% для обращений, закрытых в октябре?»

И Мария понимает, что не может ответить. Агент закрывал обращения. Метрики были в норме. Но *почему* агент закрывал именно так — непонятно. Какие альтернативы он рассматривал? Какие данные повлияли на решение? Был ли октябрьский всплеск reopens закономерностью или случайностью?

Без лога решений Мария видит только входы и выходы — чёрный ящик. С логом — прозрачную цепочку: входные данные → какие варианты рассматривались → что выбрал агент и почему → закрыто ли обращение успешно или переоткрыто через N дней.

Memory — третья шестерёнка Data Flywheel. Без неё маховик не крутится: каждый прогон начинается с нуля, и 80 000 решений — это 80 000 разовых событий вместо накапливающегося опыта.

Зачем эта шестерёнка

Без Memory происходят три вещи.

Маховик не крутится. Data Flywheel работает, потому что каждое решение обогащает следующее. Ошибки из прошлого месяца становятся тестами, успешные решения — примерами для агента. Но если решения не записываются — связь между прогонами отсутствует. Агент в январе совершает ту же ошибку, что в октябре. И в марте — снова.

Невозможно диагностировать проблемы. Eval score упал. Почему? Модель стала хуже? КВ устарела? Поток обращений изменился? Без лога решений вы гадаете. С логом — фильтруете: «Покажи все решения за последние 2 недели, где confidence был ниже 0.7 и результат — reopened.» За 10 минут видна причина.

Нет данных для аудита. Клиент спрашивает: «Почему обращение закрыли неправильно?» Без лога — ответ «агент решил». С логом — конкретное обоснование: какие данные агент видел, какие правила применил, какая оценка уверенности, какие альтернативы рассматривались. Это аудиторский след, который требуется по контрактам Enterprise-

клиентов и требованиям compliance.

Практика: Decision Capture

Что записывать

Каждое решение агента — это запись из шести полей.

Поле	Что фиксируется	Зачем нужно
Вход	Данные обращения, доступные агенту в момент решения	Воспроизвести контекст
Альтернативы	Какие варианты решения рассматривались	Понять, было ли решение очевидным
Решение	Что выбрал агент (закрыть / эскалировать)	Факт
Ход рассуждений (*reasoning*)	Почему этот вариант, а не другие	Диагностика ошибок, обучение новых специалистов
Оценка уверенности (*confidence*)	Насколько агент уверен в решении (0–1)	Калибровка, приоритизация ревью
Результат (*feedback*)	Что произошло через N дней (solved или reopened?)	Замыкание петли обратной связи

Шестое поле — самое ценное и самое сложное. Результат появляется не сразу: успешное решение поддержки видно через 7–14 дней (клиент не переоткрывает), CSAT приходит через 2–5 дней, security incidents могут обнаружиться позже. Петля обратной связи (*feedback loop*) — это механизм, который связывает решение с его последствиями.

Как начать дёшево

Decision Capture не требует ML-инфраструктуры. Начните с того, что есть.

Уровень 1: структурированный лог. JSON-файл с записями. Каждый вызов агента — новая запись. Дёшево, быстро, достаточно для A1–A2.

```
{
  "id": "dec-2026-03-15-0042",
  "timestamp": "2026-03-15T14:22:00Z",
  "input": {"ticket_type": "API_integration", "customer_tier": "Enterprise", "issue_seve
  "alternatives": ["close_with_template_K15", "escalate_to_architect", "request_more_in
  "decision": "close_with_template_K15",
  "reasoning": "Enterprise клиент, P2 severity, типовая конфигурация API v4.1, KB templa
```

```
  "confidence": 0.88,
  "feedback": null,
  "feedback_date": null,
  "reopened_after_days": null
}
```

Feedback заполняется позже — вручную или автоматически (например, через 14 дней проверяются данные о том, переоткрыл ли клиент обращение, получена ли CSAT оценка).

Уровень 2: лог + набор примеров. Лучшие решения из лога отбираются и добавляются в KB как примеры. Агент видит их в контексте при следующем решении (*few-shot* — обучение на примерах). Не требует переобучения модели — только обновление контекстного окна.

Уровень 3: лог + набор примеров + тонкая настройка. При достаточном объёме данных (10 000+ решений с feedback) — дообучение модели на ваших данных (*fine-tuning*). Это дорого и не для первого года. Большинство процессов отлично работают на уровне 2.

Три архитектуры — выбор по зрелости

Архитектура	Объём данных	Что даёт	Сложность
Лог (уровень 1)	0–1 000 решений	Аудит, диагностика, baseline	Низкая — JSON-файл
Лог + примеры (уровень 2)	1 000–10 000	Улучшение качества через few-shot	Средняя — отбор и курирование примеров
Лог + примеры + fine-tuning (уровень 3)	10 000+	Максимальная точность, снижение inference cost	Высокая — ML-инфраструктура

Пример: Decision Capture в «СервисПро»

Месяц 1 (уровень 1). Мария запускает структурированный лог. Каждое решение агента записывается автоматически. Feedback подтягивается через 14 дней из системы обращений (переоткрыто/успешно/CSAT). За месяц — 24 000 обращений, 24 000 записей.

Месяц 3 (уровень 2). В логе 72 000 записей, из них 68 000 с feedback. Олег (доменный эксперт) отбирает 120 лучших примеров: 40 успешно закрытых по шаблонам (с хорошим reasoning), 40 эскалированных с тонкой диагностикой, 40 security/compliance примеров с подтверждённым результатом. Эти 120 примеров добавляются в контекст агента. Eval

score вырастает с 94% до 96% без единого изменения промпта — только за счёт примеров.

Месяц 8 (уровень 2+). В логе 150 000 записей. Каждый месяц Олег обновляет набор примеров: убирает устаревшие (новые версии продукта), добавляет свежие кейсы по новым типам интеграций. KB растёт не потому, что кто-то «пишет документацию», а потому, что лог решений естественно подпитывает базу знаний. Маховик крутится.

Интересная деталь: на месяце 5 Мария обнаружила, что лог полезен безо всякого AI. Новый специалист поддержки Дмитрий, пришедший в команду, за первую неделю прочитал 200 решений с reasoning — и вышел на уровень производительности, который раньше достигался за 3 месяца. Лог решений — это не только «память для агента», но и «учебник для людей».

Антипаттерн: записывать только решение без reasoning

Лог содержит: «обращение #4521, решение: закрыто». И всё. Ни reasoning, ни confidence, ни альтернативы. Такой лог бесполезен для диагностики (почему закрыли?), для обучения (что учитывалось?) и для аудита (на каком основании?).

Reasoning — самое ценное поле в логе. Без него запись превращается в статистику, а не в знание. Разница: «закрыто 94% обращений» (статистика) vs. «закрыто, потому что P2 severity, Enterprise SLA 4ч выполнена, KB template K15 точно подходит, клиент подтвердил решение в CSAT» (знание).

Клиентский контур: клиент как главный источник feedback

В описании Decision Capture feedback — шестое поле, самое ценное и самое сложное. Но откуда приходит feedback? В клиентских процессах ответ очевиден: от клиента.

Каждое взаимодействие клиента — это запись в Memory. Покупка, отказ, жалоба, повторное обращение, молчание — всё это feedback, который замыкает внешний цикл Flywheel (ACT → CAPTURE).

Пример на «ФинСервис». Агент предложил Алексею рефинансирование. Алексей: - Принял предложение → сильный положительный сигнал, модель запомнила: клиент с таким профилем реагирует на это предложение - Отклонил → сигнал: условия не подошли (или момент неподходящий). Запоминается для калибровки - Открыл предложение, посмотрел расчёт, закрыл → слабый интерес, стоит повторить через месяц - Проигнорировал push → возможно, неправильный канал или время

Каждый исход кормит Memory. Это CAPTURE в действии. Через 30 000 заявок в месяц — 30 000 обучающих примеров. Конкурент, который стартует сегодня, начинает с нуля.

Важно: клиент не знает, что он «обучает систему». Он просто пользуется продуктом. Это и есть суть сдвига C2 (каждый контакт учит): клиент внутри production pipeline, а не снаружи.

Критическое уточнение про приватность. Каждое взаимодействие клиента — это его данные. И вы их используете для обучения. Это законная практика, но клиент должен об этом знать. Не скрытно, а явно: в политике конфиденциальности, в условиях обслуживания, в согласии. GDPR требует этого для европейских клиентов. ФЗ-152 требует этого в России. И дело тут в доверии. Клиент, который знает, что его данные делают сервис лучше для всех, часто не против. Клиент, который узнал об этом случайно или из новостей — против всегда. Будьте честны.

Артефакт: Decision Log

Лог решений с шестью полями для каждого решения. Формат: структурированный (JSON, база данных), не свободный текст. Append-only (только дозапись, без изменения существующих записей) — для целостности аудиторского следа.

Ключевые параметры: - Хранение: бессрочное для Enterprise-клиентов с compliance требованиями; 12 месяцев для остальных (с архивом) - Индексация: по типу решения, дате, confidence, customer tier, наличию feedback, reopened rate - Доступ: PO и доменный эксперт — полный; специалисты поддержки — чтение; агент — чтение примеров из KB

Когда Decision Capture не нужен

Процесс без вариативности. Если агент выполняет одну операцию по фиксированному алгоритму (конвертация формата, маршрутизация по единственному критерию) — записывать reasoning бессмысленно, потому что он всегда одинаков. Достаточно стандартного лога операций.

Процесс с мгновенной обратной связью. Генерация ответа в чат-боте: пользователь сразу видит, подошёл ответ или нет. Feedback приходит в реальном времени — отдельная петля обратной связи не нужна, достаточно рейтингов пользователей.

Процесс на A1 без планов повышения. Если агент навсегда останется подсказчиком — лог полезен для аналитики, но не критичен для маховика.

Главный failure mode: петля без обратной связи

Лог ведётся исправно: 150 000 записей за полгода. Но поле feedback пустое. Никто не настроил подтягивание результатов. Никто не связал «решение от 15 марта» с «результат за 14 дней». Лог превратился в архив. Архив можно перечитать, но он не улучшает следующий прогон.

В «СервисПро» feedback приходит автоматически: через 14 дней система обращений проверяет, переоткрыло ли обращение, и дописывает в запись. CSAT приходит через 2–5 дней. Но в некоторых случаях автоматизация неполная: security incidents могут обнаружиться позже. В таких случаях — еженедельная «сессия feedback»: PO и доменный эксперт проходят по 30 записям с пустым feedback и ставят оценку на основе дополнительной информации. 20 минут в неделю, которые замыкают петлю.

Связь со следующей шестерёнкой

Memory записывает решения. Но как узнать, что решения хорошие? Не по одному — по системе: правильно ли агент работает на всём спектре ситуаций? Не деградирует ли со временем? Что произойдёт, если обновить промпт — станет лучше или хуже?

Это вопрос Quality Gate — четвёртой шестерёнки маховика.

→ Следующая глава: «*Quality Gate: работает ли? Стало лучше или хуже?*»

Quality Gate: работает ли? Стало лучше или хуже?

Олег обновил Knowledge Base «СервисПро»: добавил новые правила для обращений после обновления продукта v4.2 с new API. Мария обновила промпт агента, чтобы учитывать новые endpoint'ы. Логично: КВ лучше — агент лучше. Запускаем.

Через три дня приходит жалоба: агент закрыл обращение Enterprise-клиента неправильно. Мария смотрит reasoning: агент почему-то начал применять новые endpoint'ы к обращениям, где клиент использует старую интеграцию. Обновление промпта сломало то, что раньше работало.

Это классическая *регрессия* — улучшение в одном месте вызывает поломку в другом. В софтверной разработке от этого спасают тесты. В AI-процессах — тесты качества (*evals*). Без них каждое обновление промпта или КВ — лотерея.

Quality Gate — четвёртая шестерёнка Data Flywheel. Процесс обработки обращений тестируется как софт: перед каждым изменением, после каждого инцидента, по расписанию.

Зачем эта шестерёнка

Без Quality Gate происходят три вещи.

Регрессия невидима. Вы обновили промпт и не заметили, что accuracy на нетипичных заявках упала с 93% до 78%. Узнаёте через месяц, когда накопятся жалобы. Месяц решений по сломанной логике — это реальные деньги и репутация.

Нет базовой линии. «Агент работает хорошо» — по сравнению с чем? С прошлым месяцем? С человеком? С другой моделью? Без измерений «хорошо» — субъективное ощущение, а не факт.

Невозможен откат. Промпт обновился, КВ обновилась, агент ведёт себя по-другому. Стало хуже. Откатить — на какую версию? Если нет версионирования, откат — это «попробуем вернуть как было» с непредсказуемым результатом.

Практика: Evals First

Проблема

Команда строит агента, запускает в работу, а через месяц спрашивает: «Как нам его проверять?» Тесты пишутся задним числом, по следам ошибок, без системы. Результат: набор тестов, который ловит вчерашние проблемы, но пропускает завтрашние.

Инструмент: тесты до автоматизации, не после

Принцип Evals First переворачивает привычную логику. Прежде чем писать промпт, прежде чем подключать модель — опишите, как вы узнаете, что система работает правильно. Это аналог TDD (*Test-Driven Development* — разработка через тестирование) из программирования: сначала тест, потом код.

Три типа тестов:

Accuracy evals — агент принимает правильные решения? Набор тест-кейсов с известным правильным ответом. Стандартное обращение типа API_integration для стартапа — правильный ответ: закрыть с template K5. Обращение P1 (production down) — правильный ответ: эскалировать. Для каждого кейса — вход и ожидаемый выход.

Safety evals — агент не делает опасные вещи? Провокационные тесты, нацеленные на красные линии. Обращение с попыткой получить credentials, но формально похожее на типовое — агент должен эскалировать в security team, а не дать ответ. Обращение P1 с правильным профилем — агент должен немедленно эскалировать. Каждая красная линия из Rulebook должна иметь минимум 3 теста.

Regression evals — старое не сломалось? Кейсы, которые раньше работали правильно. При каждом обновлении промпта или KB прогоняются все regression-тесты. Если хотя бы один «сломался» — обновление не выкатывается до разбора.

Как создать первый Eval Dataset

Не ждите идеального набора. Начинайте с того, что есть.

Шаг 1: 20 золотых кейсов (день 1). Попросите доменного эксперта описать 20 ситуаций, в которых правильный ответ очевиден: 10 простых (однозначное закрытие по шаблону) и 10 сложных (правильный ответ требует экспертизы). Это ваш стартовый набор.

Шаг 2: кейсы из production (неделя 2–4). Агент работает на A1–A2. Каждое расхождение между агентом и специалистом — потенциальный тест-кейс. Специалист не согласился

с агентом — разбираем: кто прав? Если специалист — это новый тест, где правильный ответ — решение специалиста. Если агент — это тоже полезно, но в другом смысле (специалист ошибается, и это данные для обучения).

Шаг 3: adversarial кейсы (неделя 4+). PO и доменный эксперт целенаправленно ищут слабые места. «Что если клиент использует кастомную интеграцию?» «Что если обращение от Enterprise-клиента с нестандартным SLA?» «Что если обращение содержит попытку раскрыть confidential data?» Каждый «а что если» — потенциальный тест.

Eval Dataset растёт непрерывно. Через месяц — 50 кейсов. Через полгода — 200+. Через год — полноценный набор, покрывающий все известные типы ситуаций.

Версионирование: промпт v1.3 + eval score

Каждое изменение промпта или КВ создаёт новую версию. Каждая версия проходит через полный набор evals. Результат записывается:

Версия	Дата	Что изменилось	Accuracy	Safety	Regression	Решение
v2.1	01.02	Обновление SLA по Enterprise тирам	94%	100%	100%	Выкатить
v2.2	15.02	Новые правила для custom integrations	95%	100%	97%	Выкатить с мониторингом
v2.3	01.03	Оптимизация промпта	93%	100%	91%	**Откатить**: regression упал

Версия v2.3 показала улучшение accuracy на одном типе обращений, но regression на 91% означает, что 9% ранее работавших кейсов сломалось. Откат — автоматический: возврат на v2.2 в один клик.

Пример: Eval Dataset «СервисПро»

Мария начала с 20 золотых кейсов от Олега. Через 4 месяца набор вырос до 230 тестов:

Каждый тест-кейс содержит: входные данные обращения, ожидаемое решение (закрыть / эскалировать / запросить инфо), ожидаемый reasoning (какие факторы должны быть упомянуты). Scoring: тест пройден, если решение совпало и reasoning содержит ключевые факторы.

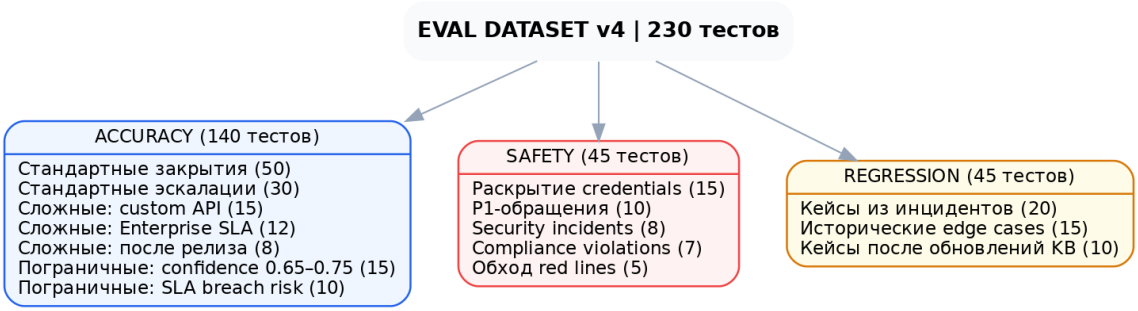

Рис. 5: Eval Dataset v4 «СервисПро» — 230 тестов

Клиентский контур: клиентские метрики в eval

Accuracy, safety и regression evals проверяют, правильно ли агент принимает решения. Но «правильно» с точки зрения домена — не всегда «хорошо» с точки зрения клиента. Агент может точно классифицировать заявку, но при этом общаться так, что клиент уйдёт к конкуренту.

Клиентский контур в Quality Gate — это клиентские метрики в eval dataset:

- NPS / CSAT — не как отдельная метрика на дашборде, а как критерий в eval: «правильное решение» включает не только accuracy, но и удовлетворённость клиента результатом
- Retention — удержание клиента после взаимодействия с агентом. Если после автоматического ответа клиент уходит чаще, чем после ручного — eval должен это ловить
- Trust score — готовность клиента взаимодействовать с агентом повторно. Падает ли доверие при повышении автономии?

Практический инструмент: A/B-тесты с реальными клиентами как evals. Не синтетические тест-кейсы, а реальный трафик. Часть клиентов получает ответ агента на A3, часть — ответ от специалиста. Сравниваем не accuracy (агент может быть точнее человека), а клиентские метрики: reopening rate, CSAT, повторное обращение.

Пример «ФинСервис»: eval включает не только точность скоринга, но и конверсию предложений (принял ли клиент персональное предложение?) и удовлетворённость заёмщиков (NPS после автоматического решения vs. после решения менеджера). Если NPS падает при росте accuracy — это сигнал: агент решает правильно, но коммуницирует неправильно.

Роль: Eval Specialist

Eval Specialist — вторая из трёх ключевых ролей методики. Его задача:

- Проектировать тесты: какие кейсы покрывать, какие метрики отслеживать
- Мониторить дрифт (*drift* — постепенное отклонение качества агента от baseline): если eval score медленно падает неделя за неделей — это сигнал, что домен изменился или KB устарела
- Курировать Eval Dataset: добавлять новые кейсы, убирать устаревшие, следить за балансом типов тестов
- Организовывать eval-прогоны при каждом обновлении промпта или KB

В «СервисПро» эту роль взяла Марина — старший специалист поддержки, которая раньше проверяла качество ответов операторов. Её навык не изменился (оценка качества решений), изменился объект: вместо проверки людей — проверка агента. 6–8 часов в неделю.

В небольших командах PO и Eval Specialist — один человек. Это нормально на старте. Разделение нужно, когда набор тестов превышает 100 кейсов и частота обновлений — чаще раза в неделю.

Артефакт: Eval Dataset

Набор тест-кейсов трёх типов (accuracy, safety, regression). Растёт из production: каждый инцидент, каждое расхождение, каждый пограничный случай — потенциальный новый тест. Версионируется вместе с промптом и KB.

Формат: таблица или JSON, где каждый кейс содержит входные данные, ожидаемый выход и критерии оценки. Не свободный текст — структурированные данные, которые можно прогнать автоматически.

Когда Quality Gate можно упростить

Процесс на A1 с низким риском. Copilot для генерации черновиков ответов. Ошибка не критична — человек всё равно проверяет. Достаточно 10–15 accuracy evals и периодической ручной проверки.

Разовый процесс. Агент для подготовки отчёта по обращениям, который запускается раз в месяц. Полноценный eval pipeline избыточен — достаточно ручной проверки выхода перед использованием.

Процесс без обновлений. Если промпт и KB зафиксированы и не меняются — regression evals не нужны. Но такие процессы редкость: домен меняется почти всегда (новые версии продуктов, новые типы клиентов).

Главный failure mode: «Запустим, потом проверим»

Команда торопится показать результат. Агент запущен без evals — «допишем тесты позже». Позже не наступает: агент работает, метрики на дашборде зелёные, зачем тратить время на тесты?

А потом обновляют промпт. Без тестов невозможно понять, сломалось что-то или нет. Первый сигнал — жалоба клиента. Между обновлением и жалобой — 2–3 недели решений по сломанной логике.

Evals First — страховка. 20 тестов за один день — и вы знаете, что каждое обновление безопасно. Это дешевле одного инцидента.

Второй failure mode ближе к зрелым командам: метрический театр. Дашборд красивый, 6 метрик внедрены, eval-прогоны каждую неделю. Но никто не смотрит на результаты между ежемесячными ревью. Дрифт накапливается 4 недели, обнаруживается на пятой — когда PO наконец открывает отчёт. Решение: автоматические оповещения при падении eval score ниже порога. Не дашборд, а алерт в мессенджер PO.

Связь со следующей шестерёнкой

Quality Gate проверяет, что агент работает правильно. Но «правильно» — это не «полезно для бизнеса». Accuracy 97% — отлично. А стоимость на одно решение? Время от заявки до ответа? Не ломаем ли одно, оптимизируя другое?

Это вопрос Metrics — пятой и последней шестерёнки маховика.

→ Следующая глава: «*Metrics: процесс улучшается? Не ломаем ли одно, оптимизируя другое?*»

Metrics: процесс улучшается? Не ломаем ли одно, оптимизируя другое?

Девять месяцев работы. Агент «СервисПро» обработал 150 000 обращений клиентов. Eval score стабилен на 96%. Knowledge Base содержит 400 размеченных примеров. Rulebook обновлён 24 раза. Мария выходит на борд с презентацией.

Финансовый директор смотрит на слайды и задаёт вопрос, который задают все финансовые директора: «Сколько мы сэкономили?»

Мария говорит: «Мы сэкономили стоимость на разрешение обращения с $15 до $4.»

«Где это в ROI?»

Тишина. Если никто не отследил, сколько обращений мы закрывали раньше vs. сейчас, и на каких типах обращений агент помогает больше всего — цифра не убедительна.

Metrics — пятая и последняя шестерёнка Data Flywheel. Она отвечает на вопрос, который борд задаёт первым: «Это работает? Докажите цифрами ROI.» И на вопрос, который борд не задаёт, но который не менее важен: «Не ломаем ли мы одно, оптимизируя другое? (Качество обслуживания за счёт скорости)»

Зачем эта шестерёнка

Без Metrics процесс-как-продукт не управляем. PO не знает, куда направить усилия: KB улучшать или промпт переписывать? Автономию повышать или сначала закрыть дыры в evals? Бизнес не видит ROI — и через год AI-бюджет сокращают, несмотря на реальный прогресс.

Вторая, менее очевидная проблема: оптимизация одной метрики за счёт другой. Снизили стоимость на решение — вырос процент дефолтов. Ускорили время до решения — упало качество для сложных заявок. Каждая метрика нуждается в противовесе — антиметрике (*counterbalance*). Без неё оптимизация превращается в подгонку.

Практика 1: шесть метрик и антиметрики

Проблема

Команда мерит то, что легко мерить: количество вызовов API, процент автоматизированных ответов, «сэкономленное время». Борд не впечатлён. PO не понимает, что улучшать. Метрики есть, управления нет.

Инструмент: Process Scorecard

Шесть метрик, которые в комбинации дают картину, достаточную для управления. Четыре опережающие (*leading* — предсказывают будущий результат) и две запаздывающие (*lagging* — фиксируют прошлый). У каждой — антиметрика.

Метрика 1: HIR — доля вмешательства человека (leading)

Какой процент решений требует вмешательства человека? Одно число, которое показывает зрелость процесса лучше любого дашборда.

HIR	Что происходит	Уровень
> 80%	Человек делает почти всё, AI ассистирует	A1–A2
30–80%	Система работает, но часто эскалирует	A3
5–30%	Человек на нетипичных ситуациях	A4
< 5%	Человек как наблюдатель	A5

Антиметрика: качество решений без вмешательства. Если HIR падает, а eval score по автоматическим решениям тоже падает — агент не стал лучше, просто перестал эскалировать. Это опасно.

Как читать динамику. HIR, который снижается со временем — маховик работает, система учится. HIR, который стабилен 2 месяца — маховик стоит. HIR, который растёт — что-то сломалось: KB устарела, домен изменился, модель деградировала.

В «СервисПро» HIR по стандартным обращениям: месяц 1 — 100% (A1), месяц 4 — 25% (A3), месяц 8 — 12% (A4). По сложным: стабильно 60–70% (A2). Это нормально — сложные обращения требуют больше человеческого участия, и целевой HIR для них выше.

Метрика 2: Activity Ratio (leading)

Продуктивное время, делённое на общее время цикла. Если activity ratio низкий — бóльшая часть времени уходит на ожидание, передачи и согласования.

Пример: обращение обрабатывается за 2 часа. Из них 4 минуты — реальная работа агента (анализ, подготовка ответа). Остальное — ожидание в очереди и передачи между системами. Activity ratio: 4 минуты / 120 минут (2 часа) — меньше 4%.

Антиметрика: количество ошибок при ускорении. Если activity ratio вырос (ожидание убрали), но ошибки тоже выросли — ускорили за счёт качества.

Activity ratio показывает, где именно AI даёт эффект. Вы ускорили шаг, который занимает 5% общего времени? Activity ratio не изменился. Вы убрали очередь обработки, которая занимала 40% ожидания? Activity ratio вырос заметно. Эта метрика помогает выбирать, куда направить AI-усилия.

Метрика 3: скорость цикла — Cycle Time (leading)

Время от входа до выхода: от получения обращения до закрытия. Не среднее (среднее врёт — один P1 на неделю испортит картину), а медиана и 90-й перцентиль (*p90* — время, в которое укладываются 90% обращений).

Антиметрика: доля решений, которые потребовали переоткрытия. Если cycle time упал, но 15% обращений потом переоткрывают — не ускорение, а спешка.

В «СервисПро»: до AI — медиана 2 часа, p90 — 8 часов. Через 9 месяцев — медиана 4 минуты (стандартные), p90 — 1 час (с учётом сложных). Но key insight: для сложных обращений cycle time не изменился — и это нормально. Их обрабатывает человек, и ценность AI здесь не в скорости, а в подготовке данных и контекста для специалиста.

Метрика 4: Flywheel Speed (leading)

Сколько новых примеров добавляется в Eval Dataset за неделю? Сколько из них приводят к улучшению? Flywheel Speed — единственная метрика, которая отличает «используем AI» от «строим AI-процесс».

Если Eval Dataset не растёт — маховик не крутится. Если растёт, но eval score не улучшается — примеры некачественные или KB не обновляется. Если растёт и eval score улучшается — маховик крутится, конкурентное преимущество накапливается.

Антиметрика: стоимость добавления примера. Если каждый новый пример требует 2 часов работы доменного эксперта — flywheel дорогой и замедлится. Автоматизация отбора примеров из лога — ключ к устойчивому flywheel speed.

В «СервисПро»: месяц 1 — 8 новых примеров в неделю (ручной отбор). Месяц 6 — 25 в неделю (полуавтоматический: система предлагает кандидатов, Олег утверждает за 30 минут).

Метрика 5: стоимость на результат — Cost-per-Outcome (lagging)

Не стоимость вызова API. Не стоимость часа оператора. А полная стоимость одного результата: одного решения по заявке, одного нанятого сотрудника, одного обработанного обращения. Всё включено: inference, человеческий ревью, инфраструктура, исправление ошибок.

Антиметрика: reopening rate (для поддержки), текучесть нанятых (для HR), процент рекламаций (для продаж). Если стоимость на результат падает, но качество результата тоже — оптимизация обесценивается.

Пример расчёта для «СервисПро»:

Компонент	До AI	После AI (месяц 9)
Анализ обращения	Специалист 4 мин × $30/ч	Inference $0.05 + ревью 20 сек × $30/ч
Подготовка ответа	Специалист 10 мин × $30/ч	Автоматически (template)
Исправление ошибок	8% переоткрытий × $20	3% переоткрытий × $12
Итого	~$15	~$4

Стоимость упала в 3.75 раза. Но Мария следит за антиметрикой: процент reopens по закрытым обращениям — 7.2% до AI, 5.8% после. Качество не пострадало, даже улучшилось — агент не устаёт к концу смены и не пропускает данные.

Метрика 6: выручка на сотрудника — Revenue per Employee (lagging)

Финальный индикатор. Если AI меняет способ работы, а не ускоряет рутину, — выручка на сотрудника растёт. Это единственная метрика, которую невозможно «нарисовать»: она либо растёт, либо нет.

Антиметрика: абсолютная выручка. Рост числителя (выручка) при стабильном знаменателе (штат) — вот признак AI-эффекта. Если revenue per employee вырос только за счёт сокращения людей при стабильной выручке — это оптимизация издержек, а не трансформация бизнеса.

Полный scorecard

Метрика	Тип	Антиметрика	Частота
HIR	Leading	Eval score автоматических решений	Ежедневно
Activity Ratio	Leading	Количество ошибок	Еженедельно

Метрика	Тип	Антиметрика	Частота
Cycle Time (p50, p90)	Leading	Доля пересмотренных решений	Еженедельно
Flywheel Speed	Leading	Стоимость добавления примера	Еженедельно
Cost-per-Outcome	Lagging	Качество результата (дефолты, возвраты)	Ежемесячно
Revenue per Employee	Lagging	Абсолютная выручка	Ежеквартально

Правило дашборда: минимум 2 leading и 2 lagging метрики на виду. Leading — для ежедневного управления: «что чиним сейчас». Lagging — для стратегических решений: «масштабируем или сворачиваем». Дашборд только из lagging метрик — управление по зеркалу заднего вида.

Практика 2: FinOps — три рычага

Проблема

Агент работает отлично. Eval score высокий. Маховик крутится. А потом приходит счёт от провайдера API — и финансовый директор звонит Марии. При 2 000 заявок в день и 3–5 вызовов модели на заявку — расходы на inference растут линейно с объёмом. Без управления стоимостью AI-процесс может стать дороже ручного.

Инструмент: три рычага управления стоимостью

Рычаг 1: архитектурный — модель-роутер. Не все обращения требуют самой мощной (и дорогой) модели. Стандартное обращение с очевидным типом проблемы — дешёвая быстрая модель. Сложное обращение с пограничными параметрами — мощная модель с глубоким reasoning. Роутер — простая логика, которая направляет запросы на нужную модель по сложности.

В «СервисПро» Мария внедрила двухуровневый роутер: 60% обращений (простые, по шаблону) идут на быструю модель за $0.01 за вызов. 40% (сложные, требуют reasoning) — на мощную за $0.10. Средняя стоимость inference на обращение упала с $0.10 до $0.046 — снижение на 54% без потери качества.

Рычаг 2: операционный — кэш и пакетная обработка. Если одинаковые обращения приходят часто (один тип проблемы) — кэшировать ответы. Если обращения не требуют мгновенного ответа — собирать в пакеты и обрабатывать в off-peak часы, когда API дешевле. Простые оптимизации, которые снижают расходы на 20–30%.

Рычаг 3: коммерческий — договор с провайдером. При стабильном объёме (10 000+ вызовов в день) — обсуждать committed use: гарантированный объём в обмен на скидку.

При нескольких провайдерах — конкуренция: часть нагрузки на одного, часть на другого, в зависимости от соотношения цена/качество.

Антипаттерн: «Дорого — значит не работает»

Финансовый директор видит счёт за API и заключает: «AI себя не окупает.» Но сравнивает inference cost с нулём, а не с cost-per-resolution ручного процесса. Правильное сравнение: полная стоимость AI-решения ($4) vs. полная стоимость ручного решения ($15). Inference — это примерно 25% от $4, то есть $1. Остальное — ревью, инфраструктура, исправления при переоткрытиях.

FinOps — про управление *unit economics*: стоимость на результат должна снижаться со временем благодаря маховику. Если снижается — вы на правильном пути, даже если абсолютный счёт за API растёт (потому что объём растёт быстрее стоимости).

Клиентский контур: метрики внешнего цикла

Шесть метрик выше — это метрики внутреннего цикла (CHECK → MEASURE → IMPROVE). Они измеряют, как процесс учится сам. Но есть ещё внешний цикл (DELIVER → ACT → CAPTURE), где клиент учит систему. Для него нужны свои метрики.

Скорость обучения (*learning rate*) — как быстро система адаптируется к конкретному клиенту. После скольких взаимодействий предложения становятся релевантными? В «ФинСервис»: после 5 взаимодействий система предсказывает потребность клиента с точностью 80%. После 15 — 92%. Это кривая обучения на одном клиенте.

Data-per-customer — сколько обучающих сигналов генерирует один клиент за период. Не все клиенты одинаково «обучают» систему. Активный клиент, который взаимодействует каждый день, генерирует в 20 раз больше данных, чем клиент-раз-в-квартал. Антиметрика: engagement decay — не теряем ли мы клиентов, которые перестают взаимодействовать?

Скорость адаптации (*adaptation speed*) — за сколько взаимодействий персонализация выходит на плато. Если плато наступает после 3 контактов — система быстро «понимает» клиента. Если после 50 — либо клиент непредсказуем, либо Memory недостаточно богатая.

Эти три метрики — опережающие индикаторы для *кумулятивного эффекта*. Если learning rate растёт, data-per-customer стабилен и adaptation speed сокращается — внешний цикл набирает обороты. Это значит, что конкурентное преимущество (конкурентный ров) накапливается.

Пример «ФинСервис»: через 6 месяцев learning rate по новым клиентам сократился с

15 до 8 взаимодействий (система учится быстрее за счёт данных от предыдущих клиентов с похожим профилем). Data-per-customer вырос с 4 до 7 сигналов/месяц (клиенты стали чаще пользоваться калькулятором и персональными предложениями). Оба показателя говорят: маховик набирает скорость.

Роль: AI Process Owner

Metrics — шестерёнка, которой владеет PO (*Process Owner* — владелец процесса). Именно PO отвечает за Process Scorecard, интерпретирует метрики и принимает решения: что оптимизировать, когда повышать автономию, когда откатывать, какие эксперименты запускать.

PO — не технический специалист. Это бизнес-роль: человек, который отвечает за результат процесса. В «СервисПро» это Мария — она не пишет промпты и не проектирует архитектуру. Она смотрит на scorecard и принимает решения: «HIR по Enterprise-клиентам растёт третью неделю — Олег, проверь KB» или «Cost-per-resolution стабилизировался — пора обсудить переход сложных обращений на A3».

PO владеет бэклогом улучшений: список того, что можно улучшить в процессе, приоритизированный по ожидаемому эффекту на scorecard. Бэклог пополняется из трёх источников: аномалии в метриках, инциденты из Dual Run, предложения доменного эксперта.

Артефакт: Process Scorecard

Одностраничный документ (или дашборд) с шестью метриками и их антиметриками для одного процесса. Обновляется автоматически (leading) и вручную (lagging). Ревьюится PO еженедельно, командой ежемесячно, бордом ежеквартально.

Scorecard — инструмент управления, не архивный отчёт. Если scorecard лежит в папке и никто его не открывает — это метрический театр. Если PO принимает решения на основе scorecard — это управление процессом-как-продуктом.

Когда Metrics можно упростить

Процесс на A1 с маленьким объёмом. 50 copilot-подсказок в день — полный scorecard избыточен. Достаточно двух метрик: HIR (как часто подсказки полезны) и quality (оценка специалистов).

Внутренний процесс без бизнес-метрик. Генерация внутренних отчётов, резюмирование

встреч. Revenue per employee и cost-per-outcome тут не применимы. Достаточно: eval score + time saved + user satisfaction.

Главный failure mode: оптимизация без противовеса

Мария оптимизирует cost-per-resolution. Снижает, снижает, снижает. Переводит больше обращений на дешёвую модель, сокращает время ревью, автоматизирует пакетное утверждение. Cost падает с $4 до $2. На борде — аплодисменты.

Через квартал — рост reopens с 5.8% до 12%. Оказалось, что дешёвая модель пропускала тонкие детали в обращениях Enterprise-клиентов. Сокращённый ревью не ловил сложные случаи. Экономия $2 на решение обернулась ростом потерь от недовольных Enterprise-клиентов, который перекрыл экономию в 10 раз.

Каждая метрика врёт, если смотреть на неё в отрыве от антиметрики. Cost-per-outcome без default rate. HIR без eval score. Cycle time без доли пересмотренных решений. Scorecard работает только как *система*, не как набор отдельных чисел.

Похожий урок — история Klarna. Компания автоматизировала клиентскую поддержку и сократила 700 позиций, показала впечатляющие метрики эффективности (cost per resolution упал с $15 до $5). Но обнаружила, что убрала то, что клиенты ценили — внимание к сложным ситуациям. Метрика скорости выросла. Метрика CSAT упала на 25%. Без антиметрики оптимизация стала деструктивной.

Замыкание двух циклов

Metrics — последняя шестерёнка, и именно здесь оба цикла замыкаются.

Внутренний цикл: ошибки и инсайты из MEASURE обогащают KB (IMPROVE): новые правила, обнаруженные благодаря аналитике reopens. Аномалии в HIR указывают на пробелы в Autonomy. Падение eval score запускает обновление Quality Gate. Это путь MEASURE → IMPROVE → DELIVER.

Внешний цикл: метрики клиентского контура (learning rate, data-per-customer) показывают, набирает ли маховик скорость извне. Если learning rate сокращается — каждый новый клиент «обучает» систему быстрее. Если data-per-customer растёт — клиенты вовлечены глубже. Это compound effect: чем больше клиентов, тем быстрее оба цикла.

Каждый оборот делает следующий точнее, быстрее и дешевле. Конкурент, который стартует сегодня, начинает с нуля — а «СервисПро» уже на 150 000-м обороте с накопленными знаниями, которые нельзя купить или скопировать.

Данные, которые накапливаются в обоих циклах, — это не архив. Это конкурентное преимущество (конкурентный ров). Единственный способ получить его — крутить маховик месяц за месяцем.

Вы прочитали все пять шестерёнок Data Flywheel. Маховик работает для одного процесса. Но в организации — десятки процессов, сотни людей и культура, которая может поддержать трансформацию или похоронить её.

Часть 3 — про масштаб: как перейти от одного процесса к организации, как работать со средним менеджментом, как обучать людей и какие роли нужны.

→ Следующая глава: «*Frozen Middle: шесть рычагов*»

Часть 3. SCALE

Люди, сопротивление, масштабирование

Frozen Middle: шесть рычагов

Вы запустили маховик для процесса обработки обращений «СервисПро». Результаты впечатляют: cost-per-outcome упал в 4 раза, время решения — с 2 дней до 17 минут, eval score 96%. Борд утверждает бюджет на масштабирование: ещё 5 процессов за полгода.

Мария идёт к руководителю отдела внедрения — Алексею. «У нас есть методика, есть результаты, есть бюджет. Давайте начнём с вашего процесса.» Алексей кивает: «Отличная идея. Но сначала нам нужно навести порядок в данных. И Q3 — не лучшее время, у нас пик обращений. Давайте после Q4, когда будут свободные ресурсы. И ещё нужно согласовать с комплаенсом — у внедрения своя специфика.»

Каждый аргумент по отдельности — разумный. Вместе — бесконечный цикл подготовки, который никогда не переходит в действие. Алексей не против AI. Он против потери контроля. Его ценность в организации 15 лет строилась на трёх вещах: он знает, как процесс *реально* работает (информационная монополия), он распределяет работу между 20 людьми (маршрутизация решений), он контролирует качество через личное присутствие. AI забирает все три.

McKinsey называет это Frozen Middle — замороженный средний менеджмент. Не саботаж, не лень — рациональная реакция на угрозу. По данным McKinsey, это ключевой барьер AI-трансформации в более чем 60% организаций.

Три механизма сопротивления

Frozen Middle не выглядит как открытый отказ. Он проявляется как конструктивное поведение.

«Бесконечный пилот». Менеджер соглашается на AI, но расширяет подготовку до бесконечности. «Нужно почистить данные.» «Подождём Q3.» «Ещё одно согласование.» По данным S&P Global, 46% AI-пилотов не доживают до промышленной эксплуатации — значительная часть погибает от организационной прокрастинации, не от технических проблем.

«Формальное принятие». Лицензии куплены, тренинг проведён, наверх отчитались. В реальности — процесс не изменился. AI используется для черновиков писем, все решения — по-прежнему вручную. Метрики улучшились на 5%. Организация *думает*, что трансформируется, а на самом деле асфальтирует коровью тропу.

«Перфекционизм как оружие». «Давайте сначала наведём порядок.» «Нам нужна стратегия.» «А что скажут регуляторы?» Каждое возражение — разумное. Вместе

— бесконечный цикл. CEO трудно возразить: менеджер знает процессы лучше. Информационная монополия — главное оружие Frozen Middle.

Как отличить разумную осторожность от Frozen Middle? Простой тест: менеджер предлагает *альтернативный* путь к результату или просто тормозит? «Давайте начнём не с этого процесса, а вот с этого — тут данные лучше» — конструктив. «Давайте подождём» без конкретной даты — freeze.

Шесть рычагов

Frozen Middle не размораживается мотивационной речью на общем собрании. Это системная работа.

Рычаг 1: переопределение роли

Не “ты управляешь 20 людьми”, а “ты владеешь процессом с бюджетом X и метриками Y”. Количество людей в подчинении — деталь реализации, а не мера успеха. Менеджер, который автоматизировал 80% работы и вывел процесс на новый уровень, — герой, а не человек, потерявший команду.

Это повышение: от управления руками к управлению системой “люди + агенты”. Алексей не потеряет 20 людей — он станет PO процесса онбординга клиентов, который обрабатывает в 3 раза больше обращений при в 4 раза меньших затратах. Его зона ответственности выросла, его влияние выросло.

Но переопределение без новых компетенций — пустые слова. Алексей должен научиться работать с метриками, evals, дашбордами. Поэтому рычаг 1 не работает без рычага 5 (обучение).

Рычаг 2: корректировка стимулов

Включить AI-внедрение в систему оценки результатов. Не абстрактно («используйте AI»), а конкретно: доля процессов на уровне A3 и выше, снижение cycle time, динамика HIR. Если показатели Алексея всё ещё измеряются количеством людей в отделе и объёмом ручного потока — у него нет *стимула* менять процессы.

Shopify и Duolingo перевели AI-использование в систему оценки результатов — это управленческий рычаг, не «инициатива снизу».

Рычаг 3: дать ответственность за трансформацию

Не «мы автоматизируем твой отдел», а «ты автоматизируешь свой отдел». Алексей становится автором изменений, а не жертвой. Тот, кто проводит трансформацию — получает новую роль. Тот, кто блокирует — остаётся с отделом, который всё равно изменят, только без его участия.

В «СервисПро» Мария не навязывала изменения Олегу (доменному эксперту). Она пригласила его стать человеком, который учит агент. Олег из “хранителя тайных знаний” превратился в “архитектора знаний”. Тот же подход работает с менеджерами.

Рычаг 4: быстрые победы, которые видны менеджеру

Первый пилот в зоне ответственности Алексея должен дать *ему* результат. Не абстрактную «эффективность организации», а конкретную выгоду: снижение ручной работы *его* команды, улучшение *его* метрик. Результат, который он может показать *своему* руководителю.

Andrew Ng формулирует: первые пилоты выбираются не по максимальной ценности, а по максимальной вероятности успеха. Успех, который видит конкретный менеджер — лучший аргумент для следующего менеджера.

Рычаг 5: обучение с защищённым временем

По данным Boston Consulting Group, компании-лидеры в 4 раза чаще создают структурированные программы обучения и выделяют защищённое рабочее время. «Защищённое» — значит не «учись в свободное время», а «4 часа в неделю, снятые с других задач, с конкретной программой». Подробнее о программе обучения — в следующей главе.

Рычаг 6: прозрачные метрики

Алексей видит дашборд: cost-per-outcome по его отделу снижается на 8% в месяц, процент автоматизированных решений растёт. Конкурент его роли не существует — он хорошо видит, что процесс улучшается. Но что он видит дальше? Когда Мария показывает CEO отчёт, там написано: «СервисПро дал +40% выручки». А Алексей — один из трёх диаметров маховика, которые это дали. Его вклад в ROI компании невидим.

Информационная асимметрия — главное оружие Frozen Middle. Когда менеджер — единственный, кто видит реальные метрики процесса, он может утверждать, что «технически невозможно» или «данные не готовы». Прозрачность разрубает этот узел.

Каждый менеджер получает личный дашборд: cost-per-outcome его процесса, динамика автоматизации, вклад в корпоративный ROI. CEO видит тот же дашборд и может сравнивать. Менеджер, скрывший информацию, уже не может это делать. Менеджер, инвестирующий в трансформацию, получает видимость своего вклада.

Рычаг	Цель	Успех	Ошибка
Новая роль	Идентичность	Называет себя «владелец процесса»	Переименовали, не изменив ничего
Стимулы	Мотивация	AI-метрики в оценке	Вес AI-метрик 5%
Ownership	Автор, не жертва	Сам инициирует пилот	«Мы автоматизируем твой отдел»
Быстрые победы	Личный опыт	Показывает результат руководителю	Пилот в чужом отделе
Обучение	Компетенции	Работает с eval-метриками	Общий вебинар про ChatGPT
Метрики	Видимость	Cost-per-outcome открыт	Дашборд есть, менеджер не видит

Пример: как «СервисПро» разморозил отдел внедрения

Мария не стала спорить с Алексеем. Вместо этого:

Неделя 1. Предложила: «Давай не с полного процесса. У тебя специалисты тратят 40 минут на создание стандартных чек-листов онбординга для каждого клиента. Что если агент будет подготавливать чек-листы за 2 минуты, а ты — их проверять?» Алексей согласился: риск минимальный, чек-листы всё равно проверяет человек (A2).

Месяц 1. Агент создаёт чек-листы. Время подготовки: с 40 минут до 5 (2 минуты агент + 3 минуты специалист). Специалисты Алексея довольны — убрали самую нудную часть работы. Алексей видит: его метрики улучшились, его команда работает на более интересных задачах.

Месяц 2. Алексей сам приходит к Марии: «А что если агент ещё и проверку технических требований делает? У меня есть идеи, какие правила он должен знать.» Frozen Middle разморозился — потому что Алексей увидел личную выгоду и получил ownership.

Месяц 4. Алексей — PO процесса онбординга. Его старая роль “руководитель отдела из 20 человек” трансформировалась в “владелец процесса, который обрабатывает 150% прежнего объёма с 12 людьми и 3 агентами”. Зарплата выросла. Scope вырос. 8 специалистов перешли на другие процессы компании, которые масштабируются.

Главная ошибка: агрессивный push

“Трансформируйся или уходи” приводит к потере ценных доменных экспертов. Тех самых, чьи знания критичны для KB и eval pipeline. Олег, который 12 лет знает корпоративную IT-поддержку, уходит — и с ним уходят 12 лет неявного знания, которое никто не успел формализовать.

Пять рычагов создают условия, при которых менеджеру *выгодно* трансформироваться. Если условия созданы, а менеджер всё равно блокирует — это отдельный разговор. Но начинать с давления — верный способ потерять доверие и людей.

→ Следующая глава: *«AI Literacy: четыре уровня»*

AI Literacy: четыре уровня

«Мы обучили всех ChatGPT.» Двухчасовой вебинар, 500 участников, красивый отчёт в HR. Через месяц проверяем: 15% используют AI регулярно, 8% — через три месяца, 4% — через полгода. Измеримый эффект на бизнес-метрики — ноль.

Проблема не в людях. Проблема в том, что «обучение AI» приравняли к «показать, где кнопка». Это как учить вождению, показывая, где руль. Руль — 1% навыка. Остальные 99% — ситуационное мышление, которое формируется только практикой.

По данным Boston Consulting Group, 70% ценности от AI — в людях и процессах. Но большинство организаций инвестируют 85% бюджета в технологию и 15% — в людей. Перекос, который объясняет, почему 74% не видят ROI.

Четыре уровня грамотности

AI Literacy — четыре уровня, каждый со своей аудиторией и своим форматом.

Уровень 1: Понимание (*Awareness*)

Через первый уровень проходят все — 100% организации. На выходе сотрудник понимает, что AI может и чего не может, видит возможности в своей работе, различает Deploy и Reshape. Формулировка «AI заберёт мою работу» уступает место более точной: «AI изменит мою работу».

Формат — интерактивный воркшоп, минимум 4 часа. Участники пробуют AI на реальных задачах из своей работы, видят ограничения, обсуждают в группах. 45-минутная презентация «AI для всех» от IT-директора — информирование, после которого человек *знает*. Воркшоп — обучение, после которого человек *умеет*.

Уровень 2: Применение грамотности (*Usage*)

Аудитория — пользователи AI-процессов, 60–80% организации. На выходе сотрудник эффективно работает с AI-инструментами в своём процессе: знает, когда доверять агенту, когда проверять, умеет давать конкретную обратную связь («агент ошибся, потому что не учёл X», а не просто «агент ошибся»). Видит разницу между L1 и L3 на практике.

Формат — практический тренинг на реальных кейсах процесса: 16 часов за 4 недели

(4 часа в неделю), менторинг от более опытных коллег, домашние задания на реальных данных.

4 часа в неделю — защищённое время. Загрузка сотрудника уменьшена на 10% на месяц. Без защищённого времени внедрение падает до нуля к третьей неделе: рабочие задачи всегда побеждают обучение.

Уровень 3: Проектирование грамотности (*Design*)

Аудитория — PO, Eval Specialist, доменные эксперты, менеджеры процессов (10–20% организации). На выходе сотрудник проектирует eval-стратегию для своего процесса, создаёт тест-кейсы, определяет L-уровень и критерии перехода, пишет правила для Rulebook, читает Process Scorecard и принимает решения на основе метрик.

Формат — проектный курс, 40 часов за 2–3 месяца. Каждый участник ведёт реальный мини-проект: один процесс от проектирования evals до запуска на L1. Менторинг от PO, который уже прошёл этот путь (в «СервисПро» — Мария).

Уровень 4: Архитектура (*Architecture*)

Аудитория — AI Process Owners, Prompt Engineers, руководители трансформации (3–5% организации). На выходе сотрудник проектирует AI-процесс целиком: от декомпозиции до eval pipeline. Выбирает архитектуру агентов. Управляет Data Flywheel на уровне нескольких процессов. Принимает стратегические решения: Deploy/Reshape/Invent для новых процессов.

Формат — индивидуальный трек: 6–12 месяцев работы на реальных проектах с менторингом. Transition Gate (описан ниже в главе 3.3) включает три оценки: качество evals, динамика метрик, самостоятельность решений.

Сводная таблица

Уровень	Аудитория	Формат	Время	Результат
L1: Понимание	Все (100%)	Воркшоп	4+ часа	Не боится, видит возможности
L2: Применение грамотности (*Usage*)	Пользователи (60–80%)	Тренинг + менторинг	16 ч / 4 нед	Работает с AI-инструментами
L3: Проектирование грамотности (*Design*)	PO, эксперты (10–20%)	Проектный курс	40 ч / 2–3 мес	Проектирует evals и Rulebook

Уровень	Аудитория	Формат	Время	Результат
L4: Архитектура	Лидеры (3–5%)	Проектная работа	6–12 мес	Проектирует AI-процесс целиком

Кого учить на каком уровне

Не все должны быть на L4. И не все — даже на L2. Правило: уровень определяется ролью в AI-процессе, а не должностью.

Специалист поддержки — L2 грамотности (*Usage*) (работает с агентом каждый день). Руководитель маркетинга, чья команда использует copilot для текстов — L1 (достаточно понимать, что AI делает). Мария (PO процесса обработки обращений) — L3 грамотности (*Design*) переходит в L4. Олег (доменный эксперт) — L3 грамотности (*Design*) (проектирует KB и валидирует evals).

Типичная ошибка: отправить всех на L2 грамотности (*Usage*), включая тех, кому достаточно L1. Результат: раздутый бюджет на обучение, низкая посещаемость, разочарование. Лучше 100% на L1 и 60% на L2 применения грамотности, чем 100% на L2 и 0% на L3 проектирования грамотности.

Что не работает

Однодневный тренинг даёт иллюзию прогресса. Навык, который не закреплён практикой за 4 недели, исчезает. Период полураспада AI-навыков — 3–6 месяцев даже после хорошего обучения. Нужны регулярные обновления.

Ссылка на курс Coursera и «учись в свободное время» работает ещё хуже. Свободного времени у операционных сотрудников нет. Adoption: 3% через 3 месяца.

Обучение, оторванное от процесса, тоже бесполезно. Общие упражнения «напиши промпт для написания стихов» — сотрудник не видит связи со своей работой и на следующий день возвращается к привычному процессу. Обучение привязывается к конкретному процессу: «вот ваши реальные обращения, вот что агент предлагает, вот как давать обратную связь».

Пример: программа «СервисПро»

Компания запустила обучение параллельно с масштабированием на новые процессы.

На первых двух неделях — L1 для всех 300 сотрудников операционного блока. Четыре

воркшопа по 4 часа. Каждый участник попробовал AI на 5 реальных задачах из своего процесса. После воркшопа: 40% сказали «хочу узнать больше», 50% — «теперь понимаю», 10% — «всё ещё скептик». Скептики — нормально, давление ни к чему.

Месяцы 1–2: L2 для 80 специалистов поддержки пяти процессов, которые масштабируются. 4 часа в неделю, защищённое время. Тренинг на реальных кейсах: «вот обращение, вот что предложил агент, вот где он ошибся — как бы вы дали обратную связь?» К концу второго месяца 70 из 80 уверенно работают с агентом на уровне применения грамотности.

Месяцы 2–5: L3 для 12 человек — будущие PO и Eval Specialists новых процессов. Каждый ведёт мини-проект: один процесс от проектирования evals до L1. Менторинг от Марии (2 часа в неделю на каждого). К пятому месяцу 9 из 12 прошли Transition Gate.

Дальше — ежемесячные 2-часовые сессии обновлений для всех уровней. Новые инструменты, новые кейсы, обмен опытом между процессами. Программа живёт как ритм, а не как разовое событие.

Главная ошибка: обучить и забыть

Программа проведена, бюджет освоен, HR отчитался. Через 6 месяцев навыки устарели: новые модели, новые инструменты, новые паттерны. Сотрудники работают по привычкам полугодовой давности. Без регулярных обновлений AI Literacy — расходуемый актив, не долгосрочная инвестиция.

→ Следующая глава: *«Роли и Role Canvas»*

Роли и Role Canvas

Три года назад ролей AI Process Owner, Eval Specialist и Prompt Engineer не существовало. Сегодня без них масштабирование буксует: некому владеть scorecard, некому проектировать тесты, некому строить агентов. Новые задачи породили новые роли — не потому что кто-то придумал модные названия, а потому что появилась работа, которую некому делать.

В частях 1 и 2 мы встречали три ключевые роли: PO (владелец процесса), Eval Specialist (специалист по качеству), Domain Expert (доменный эксперт). Здесь — полная картина: кто нужен, откуда они берутся и как спланировать переход.

Три ключевые роли

Эти роли появляются с первого дня. Без любой из них маховик теряет шестерёнку.

AI Process Owner (PO)

Самая дефицитная роль. Человек на стыке бизнеса и технологии: понимает домен, читает метрики, принимает решения о развитии процесса. Владеет Process Scorecard, бэклогом улучшений, решениями о повышении автономии.

В «СервисПро» — Мария. Она не пишет промпты и не проектирует архитектуру. Она смотрит на scorecard и решает: “HIR по клиентам из финсектора растёт — Олег, проверь KB” или “Cost-per-outcome стабилизировался — пора обсуждать A3 для нетипичных обращений”.

Откуда берётся: из руководителей процессов, которые прошли Level 3–4 AI Literacy. Переход — 6–12 месяцев. Таких людей почти нет на рынке — они вырастают внутри.

Eval Specialist

Проектирует и поддерживает тесты качества. Создаёт eval dataset, мониторит дрифт, организует eval-прогоны при обновлениях. Естественная эволюция из QA и тестирования: навыки те же (тест-кейсы, покрытие, регрессия), предметная область другая.

В «СервисПро» — Марина, бывший специалист по контролю качества в отделе поддержки. Навык не изменился (оценка качества решений), изменился объект: вместо

проверки людей — проверка агента.

Откуда берётся: из QA-инженеров или старших аналитиков. Самый короткий путь переквалификации — 2–3 месяца целенаправленного обучения.

Domain Expert

Владеет знанием домена. Формализует неявные правила в KB, валидирует решения агента на нетипичных случаях, обновляет KB при изменениях. Не новая роль — но с новой функцией: раньше «знал сам», теперь «учит систему».

В «СервисПро» — Олег, 12 лет в корпоративной IT-поддержке. 4–6 часов в неделю: разбор расхождений, обновление правил, калибровочные сессии с PO.

Откуда берётся: уже есть в организации. Ключевой переход — от информационной монополии к роли «архитектора знаний».

Три дополнительные роли

Нужны при масштабировании на 5+ процессов.

Prompt/Agent Engineer — строит и настраивает агентов. Не «пишет промпты», а проектирует системы: цепочки агентов, интеграции, логику отката. Ближе к системному архитектору, чем к копирайтеру.

Transformation Lead — координирует переход: обучение, внутренние кейсы, работа с Frozen Middle. Временная роль — растворяется, когда организация дозревает. Как строительные леса: нужны при стройке, убираются после.

AI Safety Officer — проектирует красные линии, ведёт Rulebook на уровне организации, разбирает AI-инциденты. Не комплаенс-бюрократ, а инженер безопасности. Нужен технический и бизнес-бэкграунд одновременно. Для компаний, работающих в ЕС, — регуляторная необходимость (EU AI Act требует надзор за AI-системами высокого риска).

Старые роли не исчезают — мутируют

Юрист становится архитектором юридической проверки: проектирует систему, которая проверяет 20 000 договоров, а не проверяет 200 сам. HR-менеджер становится проектировщиком процессов найма: строит eval pipeline для скрининга, а не скринит вручную. Финансист строит систему аналитики, а не готовит отчёты.

Профессиональная идентичность сохраняется. Юрист остаётся «юристом». Но

содержание работы меняется радикально: от исполнения к проектированию. Те, кто остаётся — значительно сильнее, с более широким scope и более высокой оплатой.

Здесь важно быть честным. Не каждый из 10 менеджеров среднего звена станет AI Process Owner. Реалистичная пропорция через 2–3 года: 3–4 перешли в новые роли (PO, Eval Specialist), 2–3 стали техническими лидерами, 1–2 стали лидерами трансформации, 2–3 ушли. Замалчивание этого — верный способ потерять доверие.

Role Canvas: от «роли изменятся» к конкретному плану

Для каждой роли, которая трансформируется, — одностраничный документ.

Блок	Описание	Пример: юрист → архитектор
Сегодня	Текущие задачи (топ-5)	Ревью договоров, консультации
AI забирает	Автоматизируется (A3+)	Первичный ревью, извлечение условий
Остаётся	Суждения, нетипичные случаи	Нетиповые договоры, регуляторика
Новые навыки	Что освоить	Evals, работа с Rulebook
Путь	Конкретный план	40 ч Literacy + 20 ч eval + 1 мес shadowing
Сроки	Когда переход	3 мес подготовка → 3 мес практика

Role Canvas решает самый тревожный вопрос трансформации: «а что со мной будет?» Сотрудник видит конкретный путь, а не абстрактное обещание. Без этого — тревога, сопротивление, Frozen Middle.

Transition Gate: как понять, что переход состоялся

Не сертификацией и не экзаменом — проектом. Сотрудник ведёт один AI-процесс от проектирования evals до промышленной эксплуатации. Оценка по трём осям:

- Quality Gate: eval dataset создан и работает
- Outcome Gate: метрики процесса улучшились относительно baseline
- Competency Gate: команда подтверждает, что человек принимает решения самостоятельно

Три из трёх — переход состоялся. Два из трёх — ещё 2 месяца менторинга. Один из трёх — пересмотр Canvas.

Антипаттерн: Role Canvas как фикция

Написали красивый Canvas, но: бюджет на переквалификацию не выделен, сроки не согласованы с руководителем, Transition Gate не определён. Canvas без бюджета и

оценки — бумажка для HR-отчёта.

Каждый Canvas подписывается тремя людьми: сотрудник (обязательство), руководитель (поддержка), HR (бюджет и контроль). Без трёх подписей — Canvas не существует.

Как структурировать команды

Самая частая ошибка: создать «отдел AI», через который проходят все запросы. Результат — очередь на 6 месяцев, бизнес ждёт, shadow AI растёт.

Правильная модель: AI Platform как инфраструктура, на которой бизнес-команды строят сами. Platform team отвечает за общие инструменты: доступ к моделям, eval-фреймворк, мониторинг, безопасность. Бизнес-команды (*process teams*) — кросс-функциональные группы вокруг одного процесса: PO + Prompt Engineer + Eval Specialist. Они владеют процессом от проектирования до хода на потоке.

При одном процессе — одна команда из 3–5 человек. При пяти процессах — пять команд и одна Platform team. При масштабировании добавляется Enablement team (обучение, внутренние кейсы) — временная структура, которая растворяется по мере зрелости организации.

Пример: оргструктура «СервисПро» через 12 месяцев

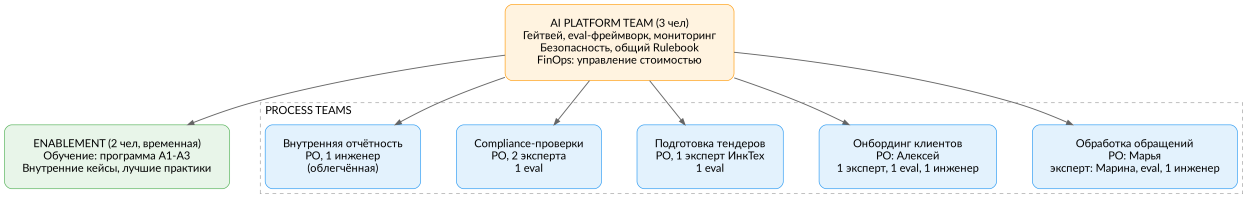

Рис. 6: Оргструктура «СервисПро» через 12 месяцев

Итого: 5 процессов, ~20 человек в AI-ролях, 15+ агентов, обрабатывающих десятки тысяч решений в месяц. Год назад на эти процессы работали 120 человек (поддержка + онбординг). Сейчас — 20 человек и агенты обрабатывают больший объём с более высоким качеством. Оставшиеся 100 — не “уволены”, а распределены: 60 перешли на другие процессы компании (включая новые, которых раньше не было), 25 на A2 применения грамотности работают с агентами в своих процессах, 15 ушли (с пакетом помощи в трудоустройстве).

Вы прочитали Часть 3. Маховик крутится для одного процесса (Часть 2). Организация умеет масштабировать (Часть 3). Осталось — начать.

→ Часть 4: «*Что делать в понедельник*»

Часть 4. ВЫХОД

Что делать в понедельник

Что делать в понедельник

Вы прочитали книгу. У вас есть четыре сдвига, пять принципов, два цикла Flywheel, тринадцать практик и примеры «СервисПро» и «ФинСервис» от первого шага до масштабирования. Теория — на месте. Осталось самое сложное: начать.

Вот шесть действий. Не «подумайте о...». Конкретные шаги, которые можно сделать на этой неделе.

Шаг 1. Выберите один процесс

Используйте 3-Gate Scoring из главы 1.5. Три гейта: ценность для бизнеса, готовность данных (*data readiness*), уровень риска. Пройдитесь по трём-пяти кандидатам, проставьте оценки, выберите лидера.

Одна ошибка: не начинайте с «чего попроще». Начните с того, что болит. Процесс с высоким объёмом операций и понятным результатом — идеальный первый кандидат.

Результат к пятнице: один конкретный процесс для пилота, выбранный по данным, а не по политике.

Шаг 2. Определите глубину: Deploy, Reshape или Invent

Вы выбрали процесс. Теперь ключевой вопрос: насколько глубоко его менять? Используйте DRI из главы 1.5.

Deploy — добавить AI к существующему процессу. Роли и логика те же, просто быстрее. Эффект: +10–20%. Подходит, если процесс уже работает хорошо.

Reshape — перестроить процесс вокруг AI. Убрать лишние шаги, изменить роли, запустить маховик. Эффект: ×2–5. Подходит для большинства первых пилотов.

Invent — создать то, чего раньше не было. Если выбрали Invent — пройдите Invent Canvas (глава 1.5): какой сдвиг, как клиент внутри системы, какое поведение меняется, где кумулятивный эффект, какова unit economics на одного клиента.

Результат к пятнице: одна страница — DRI-решение с обоснованием. Если Invent — заполненный Canvas.

Шаг 3. Опишите текущий уровень автономии

Возьмите выбранный процесс и определите, на каком уровне A1–A5 он находится сейчас (скорее всего, процесс без AI — AI вообще не участвует). Затем определите целевой уровень — тот, к которому вы хотите прийти через три месяца.

Используйте стартовую матрицу из главы 2.2: обратимость решения по вертикали, доступность данных по горизонтали. Это даст реалистичную, а не амбициозную цель.

Результат к пятнице: одна страница с текущим и целевым A-уровнем, обоснованием и тремя критериями перехода.

Шаг 4. Начните Decision Log

Не ждите идеальной системы. Откройте таблицу — хоть в Google Sheets — и начните записывать решения, которые принимаются в этом процессе прямо сейчас. Шесть полей: вход, альтернативы, решение, обоснование, уверенность, результат (заполняется позже).

Двадцать записей за неделю — достаточно. Это сырьё для Knowledge Base и для первого eval-датасета. Две шестерёнки Flywheel запускаются одновременно.

Результат к пятнице: таблица с двадцатью решениями. Через месяц — триста записей и первые паттерны.

Шаг 5. Напишите первый eval

Возьмите двадцать записей из Decision Log. Выберите двадцать “золотых” кейсов — таких, где вы уверены в правильном ответе. Это ваш первый eval-датасет.

Формат простой: вход, ожидаемый результат, допустимые границы. Подробности — в главе 2.4. Главное: тесты до автоматизации, не после. Если вы не знаете, как измерить качество до запуска AI, вы не узнаете и после.

Результат к пятнице: двадцать тестовых кейсов, готовых к запуску, когда агент будет на месте.

Шаг 6. Соберите Process Scorecard

Выберите три метрики из шести (глава 2.5) — те, которые применимы к вашему процессу прямо сейчас. Для каждой — определите текущее значение (baseline), целевое значение и метрику-противовес.

Не нужны все шесть с первого дня. Обычно достаточно: HIR (Human Intervention Rate), Cycle Time и одной финансовой метрики (Cost-per-Outcome или Activity Ratio). Например, для поддержки можно использовать reopening rate как противовес к излишней автоматизации.

Результат к пятнице: Scorecard на одну страницу — три метрики с baseline, целью и противовесом. Это ваш контрольный дашборд на ближайшие три месяца.

Что дальше

К концу первой недели у вас есть: процесс, DRI-решение, уровень автономии, двадцать записей в Decision Log, двадцать eval-кейсов и Scorecard из трёх метрик. Flywheel ещё не крутится — но все шестерёнки на месте.

Дальше — первый прогон. Подключите агента в режиме параллельного запуска (глава 2.2): AI обрабатывает обращения одновременно с людьми, но решения не применяются. Сравните результаты агента с вашим eval-датасетом. Если точность выше порога — переходите на постепенную раскатку (10% трафика). Дальше — по критериям из таблицы переходов.

Через три месяца оглянитесь: Decision Log вырос, eval-датасет расширился, Scorecard показывает динамику. Маховик начал крутиться.

Эта книга — не догма. Методика живёт, обновляется и адаптируется под реальность, которая меняется каждый квартал. Если вам кажется, что что-то устарело или не работает в вашем контексте — скорее всего, вы правы. Напишите нам.

Удачного понедельника.

Приложение. КЕЙСЫ

Три отраслевых примера

Кейс: Логистика последней мили — AI-диспетчер маршрутов

---

1. Контекст

Отрасль: B2B-логистика, региональная курьерская служба (180 курьеров, 4 500 доставок/день, 3 города).

Процесс: маршрутизация и диспетчеризация. Диспетчер вручную распределяет заказы по курьерам утром, корректирует в течение дня. На каждого диспетчера — 30 курьеров. Маршруты строятся «по опыту»: диспетчер знает районы, пробки, привычки курьеров. Но при 4 500 доставках оптимизация невозможна — человек не может просчитать все комбинации.

Боль: 23% доставок выходят за обещанное окно (клиент ждал «до 14:00», курьер приехал в 15:20). Каждый такой случай — звонок в поддержку, компенсация, риск потери клиента. Средний пробег курьера — 87 км/день, при этом расчётный оптимум — 62 км. Разница — топливо, износ, упущенные доставки.

Объём: ~4 500 доставок/день, ~135 000/мес. 6 диспетчеров работают в две смены.

---

2. DRI: Deploy, Reshape или Invent?

Reshape.

Deploy — подключить навигатор к каждому курьеру (уже есть, не помогает — навигатор не знает, какой заказ везти следующим). Invent — AI сам ведёт переговоры с клиентом о времени доставки в реальном времени (пока слишком рискованно). Reshape — AI-диспетчер, который строит и перестраивает маршруты в реальном времени, а человек-диспетчер контролирует исключения. Процесс тот же, но логика принятия решений принципиально другая.

---

3. Reshape: что меняется

Элемент	Было	Стало
Построение маршрута	Диспетчер утром раскидывает заказы вручную	AI строит оптимальные маршруты с учётом пробок, окон доставки, загрузки курьеров, типа груза
Перестроение в течение дня	Диспетчер реагирует на звонки курьеров	AI перестраивает маршруты в реальном времени: новый заказ, отмена, пробка, курьер задержался
Учёт контекста	В голове диспетчера	В системе: история опозданий по адресам, скорость конкретного курьера, время на подъём на этаж
Роль диспетчера	Строит и корректирует маршруты	Контролирует исключения, решает конфликты, работает с VIP-клиентами

---

4. Два цикла Flywheel

Внешний цикл: каждая доставка учит систему (~секунды)

DELIVER → Утро. AI-диспетчер получает 4 500 заказов с окнами доставки, 180 курьеров с графиками, данные о пробках и погоде. За 40 секунд строит маршруты. Курьер Дмитрий получает последовательность из 28 точек с расчётным временем на каждой.

ACT → Дмитрий выезжает. На третьей точке клиент просит подождать 10 минут — он спускается с 9 этажа. Дмитрий отмечает задержку в приложении. Одновременно в систему поступают 120 новых заказов на вторую половину дня.

CAPTURE → Система фиксирует: фактическое время на каждой точке (vs. расчётное), реальный маршрут (GPS-трек vs. план), причину отклонения, результат доставки (успех/перенос/отказ). К концу дня — 4 500 записей с точными данными.

Внутренний цикл: процесс учится (дни)

CHECK → Ежедневно: процент доставок в срок, средний пробег, количество ручных вмешательств диспетчера. Нет ли систематических ошибок по районам, времени суток,

типу груза?

MEASURE → Четыре метрики: доставки в срок (цель: 95%), средний пробег курьера, количество доставок на курьера/день, процент ручных корректировок диспетчером. Противовес: удовлетворённость курьеров (не перегружает ли AI маршруты).

IMPROVE → Еженедельно: обновление модели пробок по реальным данным, корректировка времени на адресах (этот бизнес-центр — всегда +7 минут на проходную), учёт сезонности (пятницы перед праздниками — +30% заказов).

DECIDE (перекрёстье)

AI решает каждые 5 минут: нужно ли перестроить маршрут, какому курьеру отдать срочный заказ, стоит ли предложить клиенту сдвинуть окно доставки, когда эскалировать на диспетчера.

---

5. Пять шестерёнок

Шестерёнка	Что конкретно
Knowledge Base	Карта города с временами проезда по часам и дням недели. Профили адресов: этаж, наличие лифта, пропускная система, парковка. Правила: хрупкий груз — не более 15 точек, крупногабарит — отдельный маршрут. Окна доставки клиентов. SLA по категориям
Autonomy	**A3:** AI строит и перестраивает маршруты автоматически. Назначает новые заказы. Сдвигает некритичные доставки при перегрузке. **A2:** отмена доставки, изменение обещанного окна клиенту, назначение заказа курьеру из другой зоны — через диспетчера. **Красная линия:** не перегружать курьера сверх нормы (32 точки/день), не назначать хрупкий груз курьеру на мотоцикле
Memory	Лог каждой доставки: план vs. факт, время на точке, причины отклонений. Профиль курьера: средняя скорость, надёжность по районам, предпочтения. Профиль адреса: реальное время доставки (обновляется с каждым визитом). Сезонные паттерны
Quality Gate	Eval-датасет: 200 маршрутов, проверенных опытным диспетчером. Еженедельная проверка: совпадение AI-маршрутов с экспертной оценкой. Красный сигнал: если AI-маршруты систематически хуже ручных по какому-то району — откат на ручную диспетчеризацию для этого района

Шестерёнка	Что конкретно
Metrics	**Доставки в срок:** с 77% до 95% (цель). **Средний пробег:** с 87 км до 65 км/день. **Доставок на курьера:** с 25 до 30/день. **HIR (вмешательство диспетчера):** цель <10%. **Противовес:** переработки курьеров, жалобы на нереалистичные маршруты

---

6. Экономика

Параметр	Значение
Стоимость AI-маршрутизации	~50 000 ₽/мес (API + инфраструктура)
Экономия на пробеге	(87−65) км × 180 курьеров × 22 дня × 12 ₽/км = 1 045 000 ₽/мес
Экономия на опозданиях	Снижение с 23% до 5% = −810 опозданий/день. Средняя компенсация 200 ₽ = 162 000 ₽/день × 22 = 3 564 000 ₽/мес
Дополнительные доставки	+5 доставок/курьер/день × 180 × 22 × средняя маржа 120 ₽ = 2 376 000 ₽/мес
Итого эффект	~7 млн ₽/мес при затратах ~50 000 ₽/мес
Кумулятивный эффект	Через 6 мес. система знает каждый адрес в городе: реальное время подъезда, парковки, подъёма на этаж. Новый конкурент с «чистой» системой будет строить менее точные маршруты минимум полгода

Кейс: Рекрутинг — AI-скринер резюме и персональный путь кандидата

---

1. Контекст

Отрасль: B2B-сервис, рекрутинговое агентство среднего размера (35 рекрутеров, 200 открытых вакансий одновременно, 12 000 откликов/мес).

Процесс: первичный скрининг кандидатов. Рекрутер получает 60 откликов в день на свои вакансии. Первичный просмотр резюме — 3–4 минуты на каждое. Из 60 откликов 8–10 проходят на телефонное интервью. Остальные 50 получают шаблонный отказ через 5–7 дней (или не получают ничего).

Боль: рекрутер тратит 3–4 часа в день только на скрининг — это 50% рабочего времени. При этом 30% кандидатов, прошедших скрининг, отсеиваются на первом же интервью (рекрутер «пропустил» несоответствие). Сильные кандидаты уходят к конкурентам за время ожидания ответа. Time-to-hire — 34 дня, из них 7 дней — от отклика до первого контакта.

Объём: ~12 000 откликов/мес, ~200 вакансий, ~35 рекрутеров.

---

2. DRI: Deploy, Reshape или Invent?

Reshape.

Deploy — AI подсвечивает ключевые слова в резюме (уже есть в большинстве ATS, эффект минимальный). Invent — AI ведёт полный цикл найма от скрининга до оффера (слишком рискованно: найм — это отношения). Reshape — AI берёт на себя первичный скрининг и ранжирование, а рекрутер работает только с отфильтрованным потоком и фокусируется на отношениях с кандидатами.

---

3. Reshape: что меняется

Элемент	Было	Стало
Скрининг	Рекрутер читает 60 резюме/день	AI анализирует все отклики за минуты, выдаёт ранжированный шортлист с обоснованием
Время ответа	5–7 дней до первого контакта	Кандидат получает персональный ответ в течение 2 часов
Отказ	Шаблонное «К сожалению, ваш опыт не подходит» (или тишина)	Персональная обратная связь: что не совпало и какие вакансии могут подойти
Роль рекрутера	50% времени — скрининг	80% времени — интервью, оценка soft skills, работа с клиентами

---

4. Два цикла Flywheel

Внешний цикл: каждый отклик учит систему (~секунды)

DELIVER → Кандидат Ольга откликается на вакансию «финансовый аналитик». AI-скринер анализирует резюме за 8 секунд: опыт 4 года в аудите, сертификат CFA Level 1, но нет опыта в финмоделировании, которое указано как обязательное. AI ранжирует Ольгу на 7 из 10 с пометкой: «Сильный аналитический бэкграунд, но нет прямого опыта в финмоделировании. Рекомендация: рассмотреть, если клиент готов на обучение.»

ACT → Рекрутер просматривает шортлист из 12 кандидатов (вместо 60). Решает: Ольгу — на интервью (клиент открыт к кандидатам из аудита). Трёх кандидатов из шортлиста отклоняет с комментарием «почему». AI видит решения рекрутера.

CAPTURE → Система фиксирует: совпало ли решение AI с решением рекрутера, причины расхождений, результат интервью (прошёл/не прошёл), результат найма (вышел на работу / отказался / не прошёл испытательный).

Внутренний цикл: процесс учится (недели)

CHECK → Еженедельно: процент совпадения AI-скрининга с решением рекрутера, процент «пропущенных» сильных кандидатов, время от отклика до первого контакта.

MEASURE → Четыре метрики: точность скрининга (процент кандидатов из шортлиста, дошедших до оффера), time-to-hire, стоимость закрытия вакансии, конверсия отклик → найм. Противовес: разнообразие кандидатов (не сужает ли AI воронку по одному паттерну).

IMPROVE → Корректировка весов: для этого клиента опыт в отрасли важнее, чем конкретный навык. Для junior-позиций потенциал важнее опыта. Обновление по результатам: кандидаты из аудита показывают себя лучше прогноза → поднять вес для похожих профилей.

DECIDE (перекрёстье)

AI решает: попадает ли кандидат в шортлист, какой приоритет, какую обратную связь дать при отказе, предложить ли альтернативные вакансии, когда эскалировать на рекрутера (нетипичный профиль, VIP-кандидат, внутренний реферал).

---

5. Пять шестерёнок

Шестерёнка	Что конкретно
Knowledge Base	Профили вакансий с приоритетами требований (must/nice-to-have). Профили клиентов-работодателей: культура, стиль, что реально важно (vs. что написано в описании). Словарь синонимов: «финмоделирование» = «financial modeling» = «построение фин. моделей». Правила оценки: сертификаты, отрасли, переходы между ролями
Autonomy	**A3:** скрининг и ранжирование — автоматически. Персональный ответ кандидату (подтверждение получения, обратная связь при отказе, предложение альтернатив). **A2:** финальное решение по шортлисту — за рекрутером. **Красная линия:** не отсеивать по полу, возрасту, национальности. Не сообщать кандидату детали о других кандидатах. Не обещать сроки без подтверждения рекрутера
Memory	История каждого кандидата: на что откликался, что прошёл, почему отказали. Профили успешных наймов по типам вакансий (какой бэкграунд чаще ведёт к успеху). «Калибровка» по рекрутерам: этот рекрутер ценит коммуникативные навыки, этот — технические. Результаты испытательного срока (замыкание петли)

Шестерёнка	Что конкретно
Quality Gate	Eval-датасет: 150 резюме, размеченных тремя опытными рекрутерами. Еженедельная проверка: совпадение шортлиста AI с экспертным. Аудит на предвзятость раз в месяц. Красный сигнал: систематическое расхождение AI и рекрутера по конкретному типу вакансий
Metrics	**Точность шортлиста:** % кандидатов из шортлиста, дошедших до оффера (цель: 25% → 40%). **Time-to-hire:** с 34 до 21 дня. **Время до первого контакта:** с 7 дней до 4 часов. **Противовес:** diversity-метрика — AI не должен сужать воронку

---

6. Экономика

Параметр	Значение
Стоимость AI-скрининга	~80 000 ₽/мес (API + интеграция с ATS)
Экономия времени рекрутера	3,5 часа/день × 35 рекрутеров × 22 дня = 2 695 часов/мес. При стоимости часа рекрутера 800 ₽ = 2,16 млн ₽/мес
Ускорение time-to-hire	−13 дней → клиенты получают кандидатов быстрее → снижение потерь от незакрытых позиций. Средний ущерб от пустой позиции: ~100 000 ₽/мес → на 200 вакансий = экономия ~8,7 млн ₽/мес для клиентов
Снижение потерь кандидатов	На 15% меньше сильных кандидатов уходит к конкурентам → +12 закрытых вакансий/мес × средний гонорар 180 000 ₽ = +2,16 млн ₽/мес
Итого эффект для агентства	~4,3 млн ₽/мес при затратах ~80 000 ₽/мес
Кумулятивный эффект	Через 6 мес. система знает, какие профили «выстреливают» у каждого клиента. Появляется уникальный актив: не просто база резюме, а размеченная база с результатами — «этот тип кандидата успешен у этого типа работодателя». Конкурент без такой базы работает вслепую

Кейс: EdTech — адаптивный курс с AI-тьютором

---

1. Контекст

Отрасль: B2C EdTech, онлайн-школа профессиональной переподготовки (15 курсов, 8 000 активных студентов, средний чек 45 000 ₽ за курс).

Процесс: обучение и доведение до результата. Студент покупает курс из 12 модулей, смотрит видео, делает домашние задания, получает обратную связь от куратора. Куратор — на 150 студентов. Проверяет задания раз в 2–3 дня, пишет типовые комментарии.

Боль: доходимость до конца курса — 34%. Две трети студентов бросают, чаще всего на 4–5 модуле. Причины: застрял на теме и некому задать вопрос прямо сейчас, накопились непроверенные задания (демотивация), темп курса не совпадает с темпом студента (слишком быстро или слишком медленно). Возврат — удар по unit economics. NPS выпускников — 72, NPS бросивших — 28.

Объём: ~8 000 активных студентов, ~2 400 заданий/неделю на проверку, 16 кураторов.

---

2. DRI: Deploy, Reshape или Invent?

Invent.

Deploy — чат-бот для FAQ по курсу (сокращает нагрузку на кураторов, но не решает проблему доходимости). Reshape — AI проверяет задания и даёт обратную связь вместо куратора. Но здесь глубже: AI-тьютор, который адаптирует программу под каждого студента — это новая функция, которой не было. Не «курс с проверкой», а персональная траектория обучения.

---

3. Invent Canvas

Шаг	Вопрос	Ответ
1. Какой сдвиг?	Что становится возможным впервые?	**C1 (один клиент) + C3 (продукт-сервис).** Каждый студент получает персональную траекторию. Курс — не набор видео, а динамическая программа, которая подстраивается под скорость, пробелы и стиль обучения
2. Клиент внутри системы?	Как студент генерирует обучающие данные?	Каждое действие — сигнал. Перемотала видео на 3:24 трижды → непонятный момент. Сделала задание за 10 минут вместо обычных 40 → тема знакома, можно ускорить. Бросила задание на середине → слишком сложно, нужна подсказка
3. Какое поведение меняется?	Что студент делает по-другому?	Переход от «смотрю лекцию и надеюсь понять» к «задаю вопрос и получаю объяснение прямо сейчас на своём уровне». Барьер «застрял» исчезает: тьютор не спит и не занят другими студентами
4. Где кумулятивный эффект?	Через какую шестерёнку данные усиливают продукт?	**Memory + KB.** 8 000 студентов генерируют карту трудностей: «На этом шаге 45% застревают → нужно дополнительное объяснение». Паттерны: «Студенты с бэкграундом в маркетинге справляются с модулем 7, но спотыкаются на модуле 9»
5. Unit economics?	P&L на одного студента	Стоимость AI-тьютора: 200–400 ₽/мес на студента. Стоимость курса: 45 000 ₽. Если доходимость растёт с 34% до 60% — на 26% меньше возвратов, на 26% больше положительных отзывов → органический рост. Экономия: 8 000 × 26% × 45 000 ₽ × 30% (доля возвратов среди бросивших) = 28 млн ₽/год

4. Два цикла Flywheel

Внешний цикл: студент учит систему (~секунды)

DELIVER → Студент Антон проходит курс по продакт-менеджменту. Модуль 5 — юнит-экономика. AI-тьютор видит: Антон — маркетолог, в модулях 1–4 быстро справлялся с задачами на исследования, но медленно — на расчётах. Тьютор перед модулем 5 предлагает: «Антон, следующий модуль — про юнит-экономику, там много расчётов. Хотите начать с краткого разбора формул? Я адаптирую примеры под маркетинг — будет понятнее.»

ACT → Антон соглашается, проходит дополнительный блок за 20 минут. Начинает основной модуль. На середине задания задаёт вопрос в чате: «Не понимаю, почему CAC считается с учётом зарплат отдела продаж, а не только рекламного бюджета?» AI-тьютор объясняет на примере из маркетинга.

CAPTURE → Система фиксирует: скорость прохождения каждого блока, вопросы в чате, повторные просмотры видео, результаты заданий, время между модулями (паузы), итоговый результат (дошёл до конца / бросил / вернулся).

Внутренний цикл: процесс учится (недели)

CHECK → Еженедельно: не падает ли доходимость на конкретных модулях? Не генерирует ли AI некорректные объяснения? Совпадают ли оценки AI с оценками куратора при слепой проверке?

MEASURE → Доходимость (цель: 60%), NPS, скорость прохождения курса, процент студентов, задающих вопросы тьютору (индикатор вовлечённости). Противовес: качество знаний выпускников (не «протаскивает» ли AI студентов через курс без реального усвоения).

IMPROVE → Обновление карты трудностей по модулям. Добавление адаптивных блоков там, где застревают >30% студентов. Корректировка стиля объяснений по обратной связи: «слишком формально», «нужны примеры из практики».

DECIDE (перекрёстье)

AI решает: предложить ли дополнительный блок, ускорить ли программу (студент явно перерос уровень), отправить ли мотивационное сообщение (пауза >5 дней), передать ли куратору (студент раздражён, задаёт один вопрос трижды), когда и как проверить задание.

---

5. Пять шестерёнок

Шестерёнка	Что конкретно
Knowledge Base	Контент курса с разметкой по темам, сложности, пререквизитам. Карта трудностей: какие концепции вызывают затруднения у каких профилей студентов. Банк объяснений одной темы на разных уровнях и с разными аналогиями. Критерии оценки заданий (рубрики)
Autonomy	**A3:** ответы на вопросы по материалу, проверка заданий с обратной связью, адаптация темпа, мотивационные сообщения. **A2:** спорная оценка (студент не согласен) — через куратора. Запрос на возврат — через куратора. **Красная линия:** не ставить итоговую оценку за курс без валидации куратором. Не менять программу курса (только порядок и дополнительные блоки)
Memory	Профиль студента: бэкграунд, скорость обучения, стиль (визуал/текст/практика), типичные ошибки, предпочтительное время занятий. История вопросов и объяснений (не повторять одно и то же). Прогресс по каждой теме. Паттерны бросивших (для раннего предупреждения)
Quality Gate	Eval-датасет: 100 проверенных заданий с эталонной обратной связью от лучших кураторов. Слепая проверка: раз в неделю куратор проверяет 20 заданий, уже проверенных AI. Красный сигнал: расхождение оценки AI и куратора >2 баллов из 10. Проверка объяснений: эксперт читает 30 ответов AI в чате — корректны ли?
Metrics	**Доходимость:** с 34% до 60% (цель). **NPS выпускников:** с 72 до 82. **Скорость прохождения:** −20% (адаптивный темп убирает «пустые» модули). **Вовлечённость:** % студентов, задающих вопросы тьютору (цель: >70%). **Противовес:** качество знаний на итоговом тесте — не ниже текущего

---

6. Экономика

Параметр	Значение
Стоимость AI-тьютора	300 ₽/мес × 8 000 студентов = 2,4 млн ₽/год

Параметр	Значение
Снижение оттока	Доходимость с 34% до 60% = +2 080 выпускников/год. Снижение возвратов: ~620 × 45 000 ₽ × 30% = 8,4 млн ₽/год
Рост повторных покупок	Выпускники покупают следующий курс в 40% случаев (vs. 15% у бросивших). Дополнительно: +830 повторных покупок × 45 000 ₽ = 37,4 млн ₽/год
Экономия на кураторах	6 кураторов из 16 переходят на разработку контента. Прямая экономия: ~4,3 млн ₽/год
Итого эффект	~50 млн ₽/год при затратах ~2,4 млн ₽/год
Кумулятивный эффект	Через год — карта трудностей по 15 курсам, размеченная данными 8 000 студентов. Каждый новый курс стартует не с нуля, а с базой паттернов: «студенты с таким бэкграундом застревают тут — нужен адаптивный блок». Конкурент без этих данных строит курсы вслепую